

Syrian Private University

Faculty of Artificial Intelligence Engineering

GRADUATION PROJECT THESIS

Syrian Private University Website

A Bilingual Laravel-Based CMS Foundation for Institutional Web Presence

Prepared by:

Hamza Almawla
Zeannab Al-Hommsi
Diaa Alsaud

Supervised by:

Dr. Mudar Kewan | Eng. Amer Khorsheed | Eng. Mohammed Al-Balkhi

Academic Year 2025 / 2026

Abstract

The Syrian Private University Website project delivers a bilingual, CMS-driven web platform foundation built on Laravel 12, Filament v3, MySQL 8, Redis, Tailwind CSS, and Alpine.js. The system serves as the institutional digital presence for Syrian Private University, enabling authorized content administrators to manage homepage sections, landing pages, navigation menus, media assets, global settings, and audit trails through a role-based admin panel.

The implemented foundation covers public bilingual rendering in Arabic (RTL) and English (LTR) with locale-aware routing, an 11-section CMS-managed homepage, a generic landing-page builder, a protected draft/preview/publish/schedule/unpublish workflow, role-based access control for super administrators, editors, and faculty-scoped editors, two-factor authentication, HTML content sanitization, SEO metadata, canonical URLs, hreflang alternates, XML sitemap generation, robots.txt, legacy redirect continuity, and Redis-ready cache behavior with tag-based invalidation.

The system architecture follows MVC with a strict service layer. Controllers remain thin, business logic resides in service classes resolved through interface contracts, and Data Transfer Objects are used to transfer structured data to view layers without exposing raw database models. The architecture was verified through automated tests using PHPUnit, including an architecture guard that enforces the service-layer rules at test time.

The automated non-property test suite passed with 395 tests and 1,169 assertions covering public rendering, admin authorization, CMS workflows, SEO behavior, cache behavior, security controls, and architecture compliance. The project achieved all six defined objectives and answers four research questions related to maintainability, bilingual support, draft safety, and quality attributes.

The project contributes a practical, extensible, and institutionally controlled CMS foundation that can serve as the basis for future full-site university modules including admissions, research, news, events, facilities, and contact/CRM workflows.

Keywords: Laravel, CMS, Bilingual Website, University Website, Filament, MVC, Service Layer, RTL/LTR, Role-Based Access Control, SEO Continuity, Arabic Content Management

Acknowledgements

We would like to express our sincere gratitude to everyone who supported us throughout the development of this graduation project.

We are deeply thankful to Dr. Mudar Kewan for his high-level project management guidance and academic oversight, which helped us define a clear and realistic project direction from the beginning.

We extend special thanks to Eng. Amer Khorsheed for his dedicated weekly supervision, technical feedback, and constructive review of our implementation progress throughout the project period. His guidance was essential in keeping the project aligned with both academic and engineering standards.

We also thank Eng. Mohammed Al-Balkhi for his monitoring support and methodology review, which helped us maintain a structured and disciplined development process.

Finally, we would like to thank Syrian Private University and the Faculty of Artificial Intelligence Engineering for providing the academic environment and resources that made this project possible.

Table of Contents

Abstract	2
Acknowledgements	3
Table of Contents	4
List of Tables	9
Chapter 1: General Introduction	10
1.1 Project Overview and Background	10
1.2 Problem Statement and Importance	10
1.3 Initial Research and Application Gap	11
1.4 Research Questions and Hypotheses	11
1.5 Project Objectives	12
1.6 Project Scope, Constraints, and Assumptions	12
1.7 Project Contribution	13
1.8 Initial Feasibility Study	13
1.9 Ethical Considerations and Data Governance	14
1.10 General Research Methodology	14
1.11 Document Structure	14
1.12 Definitions and Key Concepts	14
Chapter 2: Theoretical Framework and Literature Review	16
2.1 Theoretical Framework and Reference Concepts	16
2.1.1 University Websites as Digital Institutional Platforms	16
2.1.2 Usability and User Experience	16
2.1.3 Accessibility	17
2.1.4 Responsive Design and Mobile-First Experience	17
2.1.5 Information Architecture	17
2.1.6 Performance, Security, and SEO	17
2.2 Literature Search Methodology	17
2.3 Previous Studies and Related Work	18
2.4 Modern Technologies and Frameworks	18
2.5 Existing Systems and University Website Analysis	19
2.5.1 Current SPU Website	19
2.5.2 Regional University Benchmarks	19
2.5.3 International Benchmark	19
2.6 Systematic Comparative Analysis	20
2.7 Research Gap and Proposed Innovation	20
2.8 Modern and Future Trends	20
Chapter 3: Development Methodology and Project Management	21

3.1 Selected Development Methodology.....	21
3.1.1 Definition of Done	21
3.2 Project Timeline.....	22
3.3 Risk Management.....	23
3.4 Team Members, Supervision, and Roles	24
3.5 Quality Management.....	24
3.6 Project Management Tools	25
3.6.1 GitHub.....	25
3.6.2 Trello.....	25
3.7 Monitoring and Continuous Evaluation	25
3.8 Summary.....	25
Chapter 4: Requirements Analysis and Modeling	26
4.1 Requirements Gathering	26
4.1.1 Requirements Scope Triage.....	26
4.2 Stakeholders and Their Needs.....	26
4.3 Functional Requirements	27
4.4 Non-Functional Requirements	28
4.5 Design Constraints and Compliance	30
4.6 Requirement Prioritization Using MoSCoW	30
4.7 Requirements Modeling Using UML	30
4.7.1 Use Case Diagrams	31
4.7.2 Use Case Descriptions	33
4.7.3 Activity Diagrams.....	34
4.7.4 Sequence Diagrams	39
4.7.5 Structural and Component Diagrams.....	44
4.8 Requirements Traceability Matrix	47
4.8.1 Sources Supporting Diagram Ordering.....	47
4.9 Public Frontend Interfaces	48
.....	48
4.10 Chapter Summary.....	50
Chapter 5: Architectural and Detailed Design.....	51
5.1 High-Level System Design.....	51
5.2 Architectural Pattern.....	51
5.2.1 Rationale for MVC + Service Layer	52
5.2.2 Layer Responsibilities	52
5.3 Architectural Decision Records	53
ADR-001: Use Laravel 12 as the Backend Framework.....	53
ADR-002: Use Filament v3 for the Admin CMS	53

ADR-003: MVC with Strict Service Layer	53
ADR-004: Explicit Translation Tables	53
ADR-005: Use MySQL 8 and ADR-006: Use Redis for Production Cache.....	54
ADR-007: Protected Preview Tokens.....	54
5.4 Database Design	54
5.4.1 Core Entities	54
5.5 API and Route Design	55
5.5.1 Public Routes	55
5.5.2 Admin and Webhook Routes.....	56
5.6 Security and Authorization Design	56
5.6.1 Role Permissions.....	56
5.6.2 Threat Model (STRIDE)	56
5.7 CMS Workflow Design	57
5.8 SEO, Cache, and Continuity Design	57
5.9 DTO and View Payload Design.....	58
5.10 Chapter Summary.....	58
Chapter 6: Implementation Environment and Tools.....	59
6.1 Development Environment	59
6.2 Languages, Frameworks, and Libraries	59
6.3 Git and Branch Strategy.....	60
6.4 CI/CD Pipeline	61
6.5 Containers and Infrastructure	61
6.6 Reproducibility	62
6.6.1 Local Setup Steps.....	62
6.6.2 Composer Setup and Development Scripts.....	62
6.6.3 Testing Commands	62
6.7 Runtime Environments.....	63
6.8 Operational Commands.....	63
6.9 Chapter Summary.....	63
Chapter 7: Software Implementation	64
7.1 Project Structure	64
7.2 Public Controllers	64
7.2.1 Homepage Controller	64
7.2.2 Page Controller	65
7.3 Service Layer Implementation.....	65
7.3.1 Homepage Publishing Service	65
7.3.2 Page Service.....	66
7.3.3 Preview Service.....	66

7.3.4 Menu Service	66
7.3.5 Media Service	66
7.3.6 Audit Service	67
7.4 Admin Panel Implementation	67
7.5 Data Access Layer	67
7.6 Business Logic Layer	68
7.7 User Interface Implementation	68
7.8 Error Handling and Logs	69
7.9 Security Implementation	69
7.10 Implementation Summary	70
Chapter 8: Testing and Evaluation	71
8.1 Test Plan	71
8.1.1 Test Scope	71
8.1.2 Entry and Exit Criteria	71
8.1.3 Test Environment	71
8.2 Types of Testing	72
8.3 Test Cases	72
8.4 Test Results and Defect Analysis	74
8.5 Quality Metrics	74
8.6 Usability Testing	75
8.7 AI/ML Model Testing	76
8.8 Security Testing	76
8.9 Acceptance Decision	76
Chapter 9: Results and Analysis	78
9.1 Introduction	78
9.2 Summary of Implemented Results	78
9.3 Objective Achievement Analysis	78
9.4 Functional Results	79
9.4.1 Public Website Results	79
9.4.2 Admin Panel Results	80
9.4.3 CMS Workflow Results	80
9.4.4 SEO and Continuity Results	80
9.5 Non-Functional Results	80
9.6 Automated Testing Results	81
9.7 Research Question Answers	81
9.7.1 RQ-01: Maintainability and Governance	81
9.7.2 RQ-02: Bilingual Arabic and English Support	81
9.7.3 RQ-03: Draft Safety and Preview Workflow	82

9.7.4 RQ-04: Quality Attributes	82
9.8 Comparison With the Previous Situation	82
9.9 Limitations of the Results	82
9.10 Discussion	83
9.11 Conclusion	83
Chapter 10: Conclusion and Recommendations	84
10.1 Introduction.....	84
10.2 Project Summary	84
10.3 Main Achievements.....	84
10.4 Objective Review	85
10.5 Final Conclusions	85
10.6 Recommendations for Production Deployment	86
10.7 Recommendations for Maintenance.....	86
10.8 Future Work	87
10.9 Lessons Learned	87
10.10 Closing Statement	88
References.....	89

List of Tables

The tables in this thesis are presented in-line within each chapter. Major tables include:

- Table 1.1: Research Questions and Measurement Indicators
- Table 1.2: Project Objectives
- Table 1.3: Definitions and Key Concepts
- Table 2.1: Literature Review Summary
- Table 2.2: Comparative University Website Scoring Matrix
- Table 3.1: Project Timeline
- Table 3.2: Risk Register
- Table 3.3: Team Roles and Responsibilities
- Table 4.1: Stakeholder Analysis
- Table 4.2: Functional Requirements
- Table 4.3: Non-Functional Requirements
- Table 4.4: MoSCoW Prioritization
- Table 4.5: Requirements Traceability Matrix
- Table 5.1: Architectural Decision Records
- Table 5.2: Database Entities
- Table 5.3: DTO Catalogue
- Table 6.1: Technology Stack
- Table 6.2: CI/CD Pipeline Steps
- Table 7.1: Admin Panel Resources
- Table 8.1: Test Cases
- Table 8.2: Test Results Summary
- Table 8.3: Quality Metrics
- Table 9.1: Main Implementation Results
- Table 9.2: Objective Achievement Analysis
- Table 9.3: Non-Functional Result Summary
- Table 9.4: Comparison with Static/Manual Approach
- Table 10.1: Objective Final Status
- Table 10.2: Main Achievements

Chapter 1: General Introduction

1.1 Project Overview and Background

University websites have evolved from static informational pages into strategic digital platforms that support communication, academic visibility, student services, institutional trust, and administrative publishing workflows. For a higher education institution, the public website is often the first contact point for prospective students, current students, faculty members, researchers, parents, partners, and external visitors. It must therefore provide accurate information, multilingual accessibility, responsive presentation, search visibility, and secure content administration.

The Syrian Private University website project addresses this need by building a modern bilingual web platform for Syrian Private University. The system is implemented as a Laravel-based content management foundation that supports Arabic and English content, right-to-left and left-to-right rendering, a CMS-driven homepage, navigation management, landing-page publishing, media handling, settings management, role-based administration, and audit logging. The current project scope focuses on the homepage and administration foundation rather than the complete implementation of every future university module.

The project is technically significant because it combines public-facing university communication requirements with a controlled administrative workflow. The homepage is not hardcoded; it is managed through an admin panel and published through a draft, preview, schedule, publish, and unpublish workflow. This approach allows university editors to update institutional content without developer intervention while maintaining governance, security, cache invalidation, bilingual consistency, and traceability.

1.2 Problem Statement and Importance

University websites commonly face operational problems when content is scattered, manually updated, difficult to translate, or dependent on technical staff for routine publishing. In such environments, public information can become outdated, bilingual content may be inconsistent, and administrative changes may lack traceability. These issues directly affect user trust, student access to information, search engine visibility, and the institution's ability to communicate urgent or strategic updates.

For Syrian Private University, the problem addressed by this project is the need for a maintainable, secure, bilingual, and CMS-driven website foundation that supports the public homepage and core

publishing workflows. The website must allow authorized staff to manage homepage content, pages, navigation menus, media assets, settings, and audit logs while preventing unauthorized access, draft leakage, accidental overwrites, and uncontrolled publication.

The importance of this problem is both practical and academic. Practically, a university website must be reliable, accessible, searchable, and easy to administer. Academically, the project requires applying software engineering principles such as layered architecture, requirements traceability, data modeling, role-based access control, quality attributes, secure development, testing, and deployment readiness.

1.3 Initial Research and Application Gap

Existing website platforms and generic content management systems can provide page editing and publishing features, but they often require substantial customization to meet the specific bilingual, governance, workflow, and architectural needs of a university website. A standard CMS may support content editing but may not enforce locale-aware cache behavior, protected preview tokens, strict separation between controllers and business logic, DTO-based service returns, homepage section governance, role-specific admin access, faculty-scoped editing, and audit logging aligned with a controlled academic institution context.

The initial application gap is therefore the absence of a tailored website foundation for Syrian Private University that combines bilingual public delivery, CMS-driven homepage management, administrative governance, SEO and continuity support, secure access control, and maintainable service-layer architecture.

1.4 Research Questions and Hypotheses

The project is guided by the following research questions:

Code	Research Question	Related Goal	Answered In
RQ-01	To what extent can a custom Laravel-based CMS foundation improve the maintainability and governance of the SPU public website compared with static or manually managed publishing approaches?	G-01, G-02	Chapter 9
RQ-02	How effectively can the proposed system support bilingual Arabic and English content while preserving correct RTL/LTR rendering and locale-aware routing?	G-03	Chapter 9

Code	Research Question	Related Goal	Answered In
RQ-03	Can the system prevent unpublished draft content from becoming publicly visible while still allowing authorized preview and scheduled publishing?	G-04	Chapter 9
RQ-04	Can the implemented architecture satisfy essential quality attributes such as security, testability, SEO continuity, caching behavior, and RBAC?	G-05, G-06	Chapter 9

The main hypothesis is that a purpose-built CMS foundation using Laravel, Filament, DTOs, service-layer architecture, role-based authorization, and automated tests can provide a more maintainable and institutionally controlled website foundation than a static website or an unstructured generic implementation.

1.5 Project Objectives

Code	Objective	Verification Method
G-01	Build a maintainable public website foundation for SPU using MVC with a strict service layer and DTO-based data transfer.	Architecture review, code inspection, architecture tests
G-02	Provide an admin panel that enables authorized users to manage homepage content, pages, menus, settings, media assets, users, and audit logs.	Feature tests, admin workflow demonstration
G-03	Support bilingual Arabic and English content with correct RTL/LTR behavior and locale-aware routing.	Locale rendering tests, manual UI inspection, response header verification
G-04	Implement a safe CMS workflow that supports draft, protected preview, schedule, publish, and unpublish operations without leaking draft content publicly.	Preview tests, cache bypass checks, publish workflow tests
G-05	Implement security controls including RBAC, account locking, two-factor authentication, HTML sanitization, security headers, and audit logging.	Security feature tests, policy checks, configuration review
G-06	Preserve public discoverability and migration continuity through SEO metadata, canonical URLs, hreflang alternates, sitemap, robots.txt, legacy redirects, and unresolved request logging.	SEO tests, route checks, continuity validation commands

1.6 Project Scope, Constraints, and Assumptions

The project scope is limited to the website foundation needed for the public homepage and the administration platform. The in-scope features include the public homepage, public navigation shell,

generic bilingual landing-page builder, admin panel, homepage draft and publish workflow, menu builder, settings management, media library, audit logging, Arabic and English support, and RTL/LTR rendering.

The project does not attempt to fully implement all future university domains. Full facilities, research repository, news, events, admissions, and contact/CRM modules are outside the current implementation scope. This scope limitation is intentional because the project focuses on building the platform layer that future modules can extend.

The main technical constraints include the selected stack, compatibility with Laravel 12 and PHP 8.2 or later, MySQL 8 as the database engine, Redis for production cache behavior, Filament v3 for the admin panel, and the need to follow the architecture rules defined for the project.

1.7 Project Contribution

The project contributes a structured, secure, and bilingual CMS foundation for the Syrian Private University website. Its contribution is practical because it provides a working website platform with public pages, homepage sections, administrative management, media handling, publishing workflow, cache behavior, SEO support, redirect continuity, and audit logging.

The contribution is also engineering-focused. The implementation demonstrates how a university website can be designed using strict separation of concerns: controllers remain thin, business logic is placed in services, service contracts are defined through interfaces, DTOs are used for structured data transfer, and public service methods do not return raw database models. This makes the system easier to test, review, maintain, and extend.

1.8 Initial Feasibility Study

Technically, the project is feasible because the selected technologies are mature and widely used. Laravel provides routing, middleware, service providers, validation, authentication integration, caching, testing, and database migration support. Filament v3 provides an efficient foundation for building admin panels. MySQL 8 supports relational data storage, while Redis supports production-grade caching and tag-based invalidation.

Operationally, the project is feasible because it reduces the dependency on developers for routine content updates. Financially, the project uses open-source technologies and can be deployed on

common PHP hosting or cloud infrastructure. From a schedule perspective, the project is feasible as a graduation project because the current implementation scope is limited to the website foundation.

1.9 Ethical Considerations and Data Governance

The system handles institutional content, user accounts, media assets, settings, audit logs, and potentially contact submissions. Access to the admin panel is restricted through role-based authorization. Locked accounts cannot continue to access the system after repeated failed login attempts. Two-factor authentication support further strengthens administrative access.

Content governance is addressed through draft and preview workflows. Draft content must not be publicly visible until intentionally published. Audit logging creates traceability for administrative write operations. Privacy is addressed by avoiding unnecessary exposure of sensitive data, using environment variables for secrets, sanitizing user-supplied HTML content, and applying security headers.

1.10 General Research Methodology

This thesis follows an engineering research methodology supported by applied software development and evaluation. The research begins by identifying the institutional and technical problem, reviewing related concepts and technologies, defining requirements, designing an architecture, implementing the system, and evaluating it through tests and measurable quality indicators. The project can be classified as an applied engineering case study because it develops a real system for a specific institutional context.

1.11 Document Structure

Chapter 2 presents the theoretical framework and literature review. Chapter 3 explains the development methodology and project management process. Chapter 4 analyzes and models the system requirements. Chapter 5 presents the architectural and detailed design. Chapter 6 documents the implementation environment and tools. Chapter 7 explains the software implementation. Chapter 8 presents testing and evaluation. Chapter 9 analyzes the results and answers the research questions. Chapter 10 concludes the thesis and provides recommendations.

1.12 Definitions and Key Concepts

Term	Definition
CMS	A Content Management System that allows authorized users to create, edit, preview, publish, and manage website content.
MVC	A software architectural pattern that separates models, views, and controllers.
Service Layer	The application layer that contains business logic and coordinates data access, validation, and workflow rules.
DTO	A Data Transfer Object used to move structured data between layers without exposing raw database models.
Filament	A Laravel-based framework for building administrative panels and resources.
RTL	Right-to-left text direction used for Arabic content.
LTR	Left-to-right text direction used for English content.
RBAC	Role-Based Access Control, an authorization model that grants permissions according to user roles.
SEO	Search Engine Optimization, a set of practices that improve discoverability by search engines.
Hreflang	An HTML attribute used to declare alternate language versions of a page.
Sitemap	An XML document that lists public URLs for search engine discovery.
Audit Log	An append-only record of administrative actions used for traceability and accountability.
Preview Token	A protected token that allows authorized preview of unpublished content.
Cache Invalidation	The process of removing stale cached content after publishing or updating data.

Chapter 2: Theoretical Framework and Literature Review

This chapter establishes the theoretical and practical foundation for the Syrian Private University website project. It reviews the role of university websites, the principles of modern web usability and information architecture, accessibility and responsive design requirements, security and performance concerns, and comparative examples from Syrian, regional, and international universities. The purpose of the chapter is not only to summarize related work, but to identify the design and implementation gap that justifies the proposed SPU website foundation.

2.1 Theoretical Framework and Reference Concepts

2.1.1 University Websites as Digital Institutional Platforms

University websites have evolved from static information pages into complex digital ecosystems. A modern university website is expected to serve several audiences simultaneously: prospective students, current students, parents, faculty members, researchers, alumni, administrative staff, partners, and general visitors. This makes the university website both a communication platform and a service platform.

Lindgaard et al. (2006) showed that users form first impressions of websites within approximately 50 milliseconds. This finding is particularly important for university websites because the homepage must communicate trust and relevance almost immediately. If the visual design appears outdated, if navigation is unclear, or if the main admission pathway is hidden, prospective students may leave before engaging with the content.

2.1.2 Usability and User Experience

Usability is commonly defined as the extent to which specified users can use a system to achieve specified goals with effectiveness, efficiency, and satisfaction in a specified context of use (ISO 9241-11:2018). In university websites, usability means that a visitor can quickly identify where to apply, where to find academic programs, how to contact the university, how to switch language, and how to access services.

Nielsen (2000) identifies learnability, efficiency, memorability, error prevention, and satisfaction as essential attributes of usable systems. User experience extends usability by including emotional and perceptual dimensions. Garrett (2011) describes user experience through layers: strategy, scope, structure, skeleton, and surface. This model is relevant to the SPU website because the redesign is not only a visual update.

2.1.3 Accessibility

Accessibility ensures that websites can be used by people with different abilities and assistive technologies. The Web Content Accessibility Guidelines (WCAG 2.1) define four principles: content must be perceivable, operable, understandable, and robust (W3C, 2018). For a university website, accessibility is not optional. Universities serve diverse users, including people with visual, motor, cognitive, or temporary impairments.

2.1.4 Responsive Design and Mobile-First Experience

Responsive web design is the practice of creating layouts that adapt to different screen sizes and device capabilities (Marcotte, 2010). This is essential for university websites because a large portion of prospective students use mobile devices as their primary browsing tool. Google's mobile-first indexing policy means that the mobile version of a website directly affects search visibility (Google, 2021).

2.1.5 Information Architecture

Information architecture is the structural organization of information environments to support usability and findability (Rosenfeld, Morville, and Arango, 2015). University websites contain large and diverse content sets: faculties, departments, programs, admissions, student life, events, research, news, documents, policies, and services. Without a clear information architecture, users must search through unrelated menus and pages to complete simple tasks.

2.1.6 Performance, Security, and SEO

Website performance affects both user satisfaction and search ranking. Google's Core Web Vitals framework identifies measurable aspects of web performance such as loading, interactivity, and visual stability. Security requirements include HTTPS, password hashing, CSRF protection, rate limiting, safe file uploads, role-based authorization, audit logging, and protection against OWASP vulnerabilities. SEO practices include descriptive titles, canonical URLs, alternate language links, sitemaps, and robots.txt configuration.

2.2 Literature Search Methodology

The literature and related-work review was conducted using a mixed source strategy. Academic and professional references were selected from human-computer interaction, web usability, information architecture, accessibility, responsive design, web performance, and SEO. The review also included direct website analysis of selected universities.

Criterion	Description
Relevance	The source must address university websites, web usability, accessibility, responsive design, information architecture, SEO, or secure web platforms.
Authority	Priority was given to standards, books, peer-reviewed publications, and recognized professional references.
Recency	Recent sources were preferred for performance, SEO, and mobile-first practices. Foundational sources were retained where still widely cited.
Applicability	The source must provide concepts or practices applicable to the SPU website redesign.

2.3 Previous Studies and Related Work

The following table summarizes the most relevant references used in this chapter.

Source	Focus	Main Contribution	Relevance to SPU
Lindgaard et al. (2006)	First impressions	Users form rapid judgments of website quality within 50ms	Supports the need for a modern homepage and visual hierarchy
Nielsen (2000)	Web usability	Learnability, efficiency, memorability, error prevention, satisfaction	Supports navigation, content clarity, and task-oriented design
Nielsen and Molich (1990)	Usability heuristics	Consistency, visibility, error prevention, help, real-world match	Supports interface evaluation and admin usability
Garrett (2011)	UX design model	Strategy, scope, structure, skeleton, surface layers	Supports redesign as a full UX process
WCAG 2.1 (W3C, 2018)	Accessibility	Perceivable, operable, understandable, robust content	Supports accessible public pages and forms
Marcotte (2010)	Responsive design	Fluid layouts across devices	Supports mobile-first redesign
Google (2020, 2021)	Core Web Vitals and mobile-first indexing	Performance and mobile quality affect search visibility	Supports performance and mobile optimization
Rosenfeld et al. (2015)	Information architecture	Findability and structure in large websites	Supports audience-segmented navigation and page hierarchy

2.4 Modern Technologies and Frameworks

For the SPU website, the selected implementation stack is Laravel 12, PHP 8.2 or later, MySQL 8, Redis, Filament v3, Tailwind CSS, Alpine.js, Vite, HTMLPurifier, Google2FA, Sentry, and PHPUnit. Laravel provides a mature MVC foundation with routing, middleware, service container, database migrations, validation, caching, authentication support, and testing facilities. Filament provides an efficient framework for building the admin panel.

The project also uses a strict service-layer architecture. Controllers remain thin and delegate business logic to services through interfaces. Services return DTOs or structured payloads rather than raw Eloquent models. HTMLPurifier is used to sanitize user-supplied HTML content, reducing the risk of stored cross-site scripting in CMS-managed areas.

2.5 Existing Systems and University Website Analysis

The comparative analysis examined SPU and six other university websites: University of Kalamoon, Arab International University, American University of Sharjah, Qatar University, KAUST, and Stanford University. The analysis used seven dimensions: visual design, navigation and structure, homepage layout, content strategy, functional elements, strengths, and weaknesses.

2.5.1 Current SPU Website

The current SPU website provides basic institutional information and maintains a recognizable identity through dark blue and burgundy colors. It also offers multilingual support across Arabic, English, French, Spanish, and German. However, the visual style is traditional, the homepage is dense, and the hero carousel lacks a clear strategic message. Navigation does not segment visitors by audience, and there is no strong application CTA or homepage statistics.

2.5.2 Regional University Benchmarks

The Arab International University includes quick links, bilingual navigation, a program search tool, and ranking badges. The American University of Sharjah provides strong audience segmentation, high-quality photography, a persistent Apply Now CTA, and clear employment outcome statistics. Qatar University presents structured audience pathways and strong news presentation. KAUST uses bold statistics, excellent visual hierarchy, and a narrative linking research and innovation.

2.5.3 International Benchmark

Stanford University represents an international benchmark for university website design. It uses editorial storytelling, generous white space, refined typography, mission-centered messaging, human photography, and audience-specific navigation. Its structural logic can inform SPU homepage planning: mission, academics, research, campus life, admissions, and community.

2.6 Systematic Comparative Analysis

University	Visual Design	Navigation	Hero Section	CTAs	Mobile	Content	Research
SPU Current	2/5	3/5	2/5	1/5	2/5	2/5	1/5
Univ. of Kalamoon	3/5	2/5	3/5	2/5	3/5	3/5	4/5
Arab Intl. Univ.	3/5	4/5	3/5	4/5	3/5	3/5	3/5
AUS	5/5	5/5	4/5	5/5	5/5	4/5	4/5
Qatar University	4/5	4/5	4/5	4/5	4/5	4/5	4/5
KAUST	5/5	5/5	5/5	5/5	5/5	5/5	5/5
Stanford University	5/5	5/5	5/5	5/5	5/5	5/5	5/5

2.7 Research Gap and Proposed Innovation

The analysis identifies a clear gap between the current SPU website and the expectations of a competitive private university website. The gap is not limited to visual appearance. It includes information architecture, audience segmentation, admissions conversion, credibility communication, mobile experience, content governance, and administrative maintainability.

The proposed project addresses this gap by building a CMS-driven bilingual website foundation. Unlike a static redesign, the project provides an administrative platform that allows authorized editors to manage homepage sections, pages, menus, settings, media assets, users, and audit logs. The innovation is therefore practical and architectural: adapting best practices from leading university websites to the SPU context through a maintainable Laravel and Filament foundation.

2.8 Modern and Future Trends

Several future trends are relevant to university websites. First, personalization may adapt content based on visitor type or language preference. Second, AI-assisted search and chatbot support may help students find information faster. Third, structured data and semantic SEO will become more important as AI systems consume institutional content. Fourth, accessibility and performance expectations will continue to rise. Finally, university websites increasingly operate as integrated digital ecosystems with admissions workflows, research repositories, news systems, events calendars, and alumni portals.

Chapter 3: Development Methodology and Project Management

This chapter explains the methodology used to manage and implement the Syrian Private University website project. The project started on March 1, 2026, under the project management supervision of Dr. Mudar Kewan and work-monitoring supervision by Eng. Amer Khorsheed and Eng. Mohammed Al-Balkhi. The project adopted a hybrid methodology that combines Waterfall planning with Agile incremental implementation.

3.1 Selected Development Methodology

Phase	Approach	Purpose
Phase 1	Waterfall - Planning and Documentation	Understand the project scope, gather and analyze requirements, define the system architecture, and prepare a stable product backlog before implementation.
Phase 2	Agile - Incremental Implementation	Build the system through time-boxed increments, review each increment, refine the backlog, and prepare the system for launch readiness.

The methodology follows Rolling Wave Planning. The overall project direction and milestones were defined early, but each sprint was planned in detail only shortly before execution. This reduced the risk of over-planning uncertain implementation details while preserving a clear academic and engineering structure.

3.1.1 Definition of Done

A task was considered complete only when it satisfied the following criteria:

Criterion	Description
Functional completion	The requested behavior was implemented according to the requirement or backlog item.
Architecture compliance	Controllers remained thin, business logic was placed in services, and service contracts were used where required.
Bilingual behavior	Public content and admin fields supported Arabic and English where applicable.
Security review	Access control, validation, sanitization, and authorization were considered for the feature.
Testing	Relevant unit, feature, or manual tests were completed.

Criterion	Description
Review	The task was reviewed by another team member or discussed during the weekly supervision meeting.
Documentation	Any important setup, workflow, or decision was documented in the repository or thesis notes.

3.2 Project Timeline

Period	Phase	Main Activities	Main Outputs	Status
Week 1: Mar 1-7, 2026	Planning	Study current SPU website, identify target audiences, review comparable universities, define project boundaries	Initial analysis, audience list, redesign direction	Completed
Week 2: Mar 8-14, 2026	Requirements & Architecture	Define functional and non-functional requirements, select stack, define bilingual strategy, define roles	Requirements draft, architecture direction, product backlog	Completed
Week 3: Mar 15-21, 2026	Sprint 1	Build foundation: Laravel setup, database migrations, authentication, role model, homepage structure	Working project foundation and initial public shell	Completed
Week 4: Mar 22-28, 2026	Sprint 1 Review	Review foundation, refine backlog, improve layout and admin workflow assumptions	Updated backlog, sprint review notes	Completed
Week 5: Mar 29-Apr 4, 2026	Sprint 2	Implement CMS workflows: draft/publish, page management, menu builder, media library, settings	Functional admin CMS features	Completed
Week 6: Apr 5-11, 2026	Sprint 2 Review	Review admin usability, bilingual rendering, menu behavior, and media handling	Updated issues list and refinements	Completed
Week 7: Apr 12-18, 2026	Sprint 3	Implement SEO, sitemap, redirects, cache behavior, preview safety, audit logging, launch validation	Launch-oriented website foundation	Completed
Week 8: Apr 19-30, 2026	Final Integration & Testing	Run tests, fix defects, verify launch readiness, prepare	Tested v1.0 foundation,	Completed

Period	Phase	Main Activities	Main Outputs	Status
		documentation and thesis evidence	documentation, thesis material	

3.3 Risk Management

Code	Risk	Probability	Impact	Strategy	Owner
R-01	Requirements expansion beyond the homepage and CMS foundation	High	High	Mitigate: Freeze scope around homepage, navigation, admin foundation, media, settings.	Full team
R-02	Bilingual Arabic/English behavior becomes inconsistent	Medium	High	Mitigate: Use explicit locale-specific storage, URL prefixes, and RTL/LTR handling.	Hamza, Diaa
R-03	Draft content becomes visible publicly	Low	High	Avoid: Separate draft/published states, protect previews with tokens, verify through tests.	Hamza
R-04	Admin users overwrite each other's changes	Medium	Medium	Mitigate: Use version-based optimistic locking.	Diaa
R-05	Poor mobile usability affects prospective students	Medium	High	Mitigate: Design responsive layouts from the beginning.	Zaineb
R-06	Security vulnerabilities in admin or CMS inputs	Medium	High	Avoid: Apply RBAC, CSRF protection, HTML sanitization, safe file upload rules.	Hamza, Diaa
R-07	Performance problems due to uncached public pages	Medium	Medium	Mitigate: Use Redis-compatible cache strategy with invalidation on publish.	Diaa
R-08	Lack of deployment clarity for future handoff	Medium	Medium	Mitigate: Maintain README, launch validation commands, operational documentation.	Full team

Code	Risk	Probability	Impact	Strategy	Owner
R-09	Delays caused by limited development time	High	Medium	Mitigate: Deliver foundation features first, defer full-site modules.	Full team

3.4 Team Members, Supervision, and Roles

Name	Role	Responsibilities
Dr. Mudar Kewan	Supervisor / Project Manager	High-level project management, academic guidance, and approval of the general project direction.
Eng. Amer Khorsheed	Supervisor	Work monitoring, weekly progress follow-up, technical feedback, and implementation review.
Eng. Mohammed Al-Balkhi	Supervisor	Work monitoring, methodology review, and implementation feedback.
Hamza Almawla	Backend Developer / Architecture and Services	Laravel backend, service-layer structure, contracts, DTOs, CMS workflows, authentication, preview/publish behavior, audit logging, and backend tests.
Diaa Alsaud	Backend Developer / Data and Integration	Database migrations, models, admin integration, cache behavior, SEO/continuity features, command-line tooling, route behavior, and backend validation.
Zaineb Alhomsy	Frontend Developer / Public Interface	Public homepage layout, responsive interface, RTL/LTR visual behavior, Tailwind styling, page presentation, and visual consistency.

3.5 Quality Management

Practice	Application in the Project
Architecture rules	Controllers remain thin, services contain business logic, interfaces are used for dependencies, DTOs are returned from public service methods.
Code review	Implemented tasks were reviewed by another team member before being considered finished.
Automated tests	PHPUnit feature and unit tests verify admin access control, homepage workflow, SEO, redirects, cache, and service logic.
Manual review	Public pages and admin workflows were reviewed manually for usability, bilingual behavior, and responsive layout.
Security checks	Role policies, account lockout, CSRF protection, HTML sanitization, file validation, and audit logging were considered during implementation.

Practice	Application in the Project
Launch readiness	Custom validation commands and checklist documents were used to prepare the project for handoff and deployment.

3.6 Project Management Tools

3.6.1 GitHub

GitHub was used to store source code, configuration files, documentation, and implementation history at <https://github.com/Zainab19hm/Spu-Website>. The adopted branch strategy followed a practical student-project workflow:

Branch Type	Purpose
`main`	Stable project version.
`develop`	Active integration branch for ongoing work.
`feature/*`	Feature-specific implementation branches.
`hotfix/*`	Urgent fixes when needed.

3.6.2 Trello

Trello was used to track tasks and backlog status. The board contained: Backlog, To Do, Doing, In Review, Done, and Change Log lists. Examples of tracked cards include system design architecture, activity diagrams, RTM, requirements database, use case specification, database migrations, ERD diagram, frontend hero/navbar/static implementation, and backend service/database tasks.

3.7 Monitoring and Continuous Evaluation

The project was monitored through weekly meetings with Eng. Amer Khorsheed. These meetings were used to review progress, clarify requirements, discuss technical decisions, evaluate completed work, and adjust the backlog for the following week. Dr. Mudar Kewan provided high-level project management supervision.

3.8 Summary

The hybrid methodology was suitable for the SPU website project because it balanced planning discipline with implementation flexibility. The Waterfall phase provided the requirements foundation, architectural direction, and product backlog. The Agile phase enabled the team to deliver the system incrementally through three main sprints. The project was completed over an approximate two-month schedule starting on March 1, 2026.

Chapter 4: Requirements Analysis and Modeling

This chapter defines what the Syrian Private University website system must do, who uses it, what quality constraints it must satisfy, and how the requirements are traced to design and testing. The requirements are written according to the principles of ISO/IEC/IEEE 29148:2018, which states that good requirements should be necessary, verifiable, unambiguous, complete, correct, and traceable.

4.1 Requirements Gathering

The requirements were gathered through multiple sources: studying the existing SPU website, reviewing comparable university websites, relying on supervision discussions with Eng. Amer Khorsheed, using the approved requirements database SPU Official Website - Requirements Database v5.0, and analyzing implementation constraints.

4.1.1 Requirements Scope Triage

Group	Description	Treatment in This Thesis
Implemented foundation requirements	Bilingual homepage, admin CMS, menu builder, media library, settings, preview/publish workflow, RBAC, audit logging, SEO metadata, sitemap, redirects, cache behavior	Fully analyzed and traced in this chapter
Foundation-supported future requirements	About, admissions, facilities, research, campus life, e-services, news, contact landing pages	Treated as supported future content structures
Deferred full-domain requirements	CRM lead dashboard, full admissions processing, expert finder, full research repository, analytics KPI dashboard, full facilities module	Listed as future work

4.2 Stakeholders and Their Needs

Stakeholder	Main Needs	Priority
Prospective students	Discover SPU, explore faculties/programs, understand admissions path, switch language, use mobile interface	High
Current students	Access announcements, links, pages, services, and university information	High
Parents and general visitors	Understand university credibility, faculties, achievements, contact information	Medium
Faculty and staff	Access relevant institutional pages and public academic information	Medium
Editor	Manage homepage, pages, menus, media, and settings without coding	High

Stakeholder	Main Needs	Priority
Faculty editor	Manage scoped pages and media related to an assigned faculty	Medium
Super administrator	Manage all content, users, roles, audit logs, settings, and operational controls	High
SPU IT team	Deploy, maintain, backup, monitor, and troubleshoot the system	High

4.3 Functional Requirements

ID	Requirement Description	Priority	Status
SW-FR-001	The system shall redirect the root URL `/` to the default Arabic homepage `/ar`.	Must	Implemented
SW-FR-002	The system shall render the public homepage in Arabic and English using locale-specific URLs `/ar` and `/en`.	Must	Implemented
SW-FR-003	The system shall render Arabic pages with RTL direction and English pages with LTR direction.	Must	Implemented
SW-FR-004	The homepage shall consist of 11 fixed CMS-managed sections: hero, hero_stats, academic_faculties, achievements_highlights, choose_your_path, university_news, research_studies, events_activities, medical_facilities_services, bottom_stats, and footer.	Must	Implemented
SW-FR-005	Authorized editors shall be able to save homepage content as a draft without changing the public published homepage.	Must	Implemented
SW-FR-006	Authorized editors shall be able to preview unpublished content through a protected tokenized preview URL.	Must	Implemented
SW-FR-007	Authorized editors shall be able to publish homepage content after validation of required content.	Must	Implemented
SW-FR-008	Authorized editors shall be able to schedule future publishing and unpublish existing content.	Should	Implemented
SW-FR-009	The system shall provide a generic bilingual landing-page builder for top-level pages.	Must	Implemented
SW-FR-010	The system shall provide admin authentication for access to `/admin` routes.	Must	Implemented
SW-FR-011	The system shall enforce role-based access control for `super_admin`, `editor`, and `faculty_editor`.	Must	Implemented
SW-FR-012	The system shall restrict user management and audit log visibility to `super_admin`.	Must	Implemented
SW-FR-013	Faculty editors shall only access pages and media matching their assigned `faculty_scope_slug`.	Must	Implemented

ID	Requirement Description	Priority	Status
SW-FR-014	The menu builder shall allow authorized editors to manage header, footer, and utility menus with locale-aware URLs.	Must	Implemented
SW-FR-015	The menu builder shall enforce a maximum menu depth of 2 levels.	Should	Implemented
SW-FR-016	The media library shall allow authorized users to upload, search, filter, edit metadata, and soft-delete media assets.	Must	Implemented
SW-FR-017	Media metadata shall support Arabic and English title, alt text, and captions.	Must	Implemented
SW-FR-018	The settings page shall allow authorized editors to manage utility navigation, footer, emergency notice, contact, social, and SEO default settings.	Must	Implemented
SW-FR-019	The system shall create audit log entries for admin write operations.	Must	Implemented
SW-FR-020	The public site shall render SEO metadata including title, description, canonical URL, hreflang alternates, and robots directives.	Must	Implemented
SW-FR-021	The system shall generate a public XML sitemap containing only published and enabled pages.	Must	Implemented
SW-FR-022	The system shall provide `robots.txt` with environment-appropriate indexing behavior.	Must	Implemented
SW-FR-023	The system shall resolve exact and pattern-based legacy redirects and log unresolved legacy requests.	Should	Implemented
SW-FR-024	The system shall provide launch validation and cache warm commands for operational readiness.	Should	Implemented

4.4 Non-Functional Requirements

ID	Quality Category	Metric	Acceptance Criterion
SW-NFR-001	Functional suitability	Published content correctness	Public homepage and published pages render only published data; drafts do not appear publicly
SW-NFR-002	Performance efficiency	Public response caching	Public page cache keys include locale and return `X-Cache` as HIT, MISS, or BYPASS
SW-NFR-003	Performance efficiency	Cache invalidation	Homepage, page, menu, and settings updates invalidate relevant cache groups
SW-NFR-004	Compatibility	Browser/device behavior	Public pages remain usable on desktop and mobile layouts

ID	Quality Category	Metric	Acceptance Criterion
SW-NFR-005	Usability	Navigation clarity	Header, footer, and utility navigation are manageable and no deeper than two levels
SW-NFR-006	Reliability	Preview safety	Draft content is accessible only through valid preview tokens and never through normal public routes
SW-NFR-007	Security	Admin access protection	Admin routes require authentication and enforce role-based permissions
SW-NFR-008	Security	Account protection	User accounts lock after 5 failed login attempts
SW-NFR-009	Security	Input sanitization	User-supplied HTML content is sanitized using an allowlist
SW-NFR-010	Security	Transport and headers	Public/admin responses include security headers such as X-Frame-Options and X-Content-Type-Options
SW-NFR-011	Maintainability	Architecture compliance	Controllers must not query Eloquent directly; services contain business logic; interfaces do not return raw models
SW-NFR-012	Maintainability	DTO usage	Public service methods return DTOs, collections, bool, paginator, or structured payloads, not raw Eloquent models
SW-NFR-013	Portability	Environment setup	System can be installed using Composer, npm, migrations, seeders, and environment configuration
SW-NFR-014	Accessibility	Semantic page structure	Public pages include correct language/direction attributes and accessibility landmarks
SW-NFR-015	Search visibility	SEO completeness	Canonical URLs are absolute and locale-correct; hreflang alternates are reciprocal; sitemap excludes draft URLs
SW-NFR-016	Internationalization	Bilingual content storage	CMS-managed content stores Arabic and English independently without machine translation fallback
SW-NFR-017	Governance	No draft leakage	Draft, scheduled, disabled, admin, and preview URLs are excluded from sitemap and public publishing flows
SW-NFR-018	Security	MFA support	Admin accounts support multi-factor authentication through TOTP challenge flow

4.5 Design Constraints and Compliance

Constraint	Description	Impact on Requirements
Technology stack	Laravel 12, PHP 8.2+, MySQL 8, Redis, Filament v3, Tailwind CSS, Alpine.js, Vite	Requirements must be implementable within the Laravel and Filament ecosystem.
Bilingual support	Arabic is the default locale and English is secondary	All CMS-managed content must support Arabic and English where applicable.
Directionality	Arabic uses RTL and English uses LTR	Public layouts and response headers must reflect the active locale direction.
Service-layer architecture	Business logic must live in services, not controllers or models	Requirements must be implemented through contracts, services, and DTOs.
Admin governance	Different users have different permissions	RBAC and faculty scoping are mandatory requirements.
Draft safety	Unpublished content must not leak publicly	Preview and cache behavior must separate draft and published data.
Migration continuity	Old URLs may still be accessed by users/search engines	Legacy exact and pattern redirects are required.

4.6 Requirement Prioritization Using MoSCoW

Priority	Requirements	Rationale
Must Have	Public AR/EN homepage, RTL/LTR rendering, admin authentication, RBAC, fixed homepage sections, draft/preview/publish workflow, page builder, menu builder, media library, settings, audit logs, SEO metadata, sitemap	Required for a working website foundation and safe CMS operation.
Should Have	Scheduled publishing, unpublish workflow, legacy redirects, cache warming, launch validation, faculty-scoped editing, two-factor authentication	Strengthen operational readiness, security, and governance.
Could Have	Advanced search, contact/lead management, full admissions workflow, full news/events/research modules	Useful extensions but outside the current foundation scope.
Won't Have This Time	Complete CRM, full research repository, full facilities module, full admissions module, complete events/news engines	Explicitly deferred to future development phases.

4.7 Requirements Modeling Using UML

The diagrams are placed in this chapter using a progressive modeling order: first the use case diagrams define actors, system boundaries, and functional scope; then activity diagrams describe workflow behavior; sequence diagrams then show time-ordered interactions between runtime participants; finally, the class and component diagrams present the structural and architectural views of the system. This ordering follows UML

documentation practice, where UML is used to visualize, specify, construct, and document software artifacts, and aligns with software design documentation principles that separate behavioral views from structural and architectural views (Object Management Group, 2017; IEEE Computer Society, 2009; ISO/IEC/IEEE, 2022).

4.7.1 Use Case Diagrams

The main use case diagram includes the following actors and their primary use cases:

Actor	Main Use Cases
Public Visitor	View homepage, switch language, browse page, use navigation, view SEO-indexed content
Prospective Student	Explore faculties, follow CTA, access admissions-related page, contact university
Editor	Manage homepage, manage pages, manage menus, manage media, manage settings, preview content, publish content
Faculty Editor	Manage scoped pages, manage scoped media, preview scoped content
Super Administrator	Manage users, assign roles, unlock accounts, view audit logs, manage all CMS resources
SPU IT Maintainer	Run launch validation, warm cache, validate redirects, monitor unresolved legacy requests

Figure 4.1: High-Level Use Case Diagram

High-level use case view showing the main external actors and major capabilities of the SPU Website Foundation.

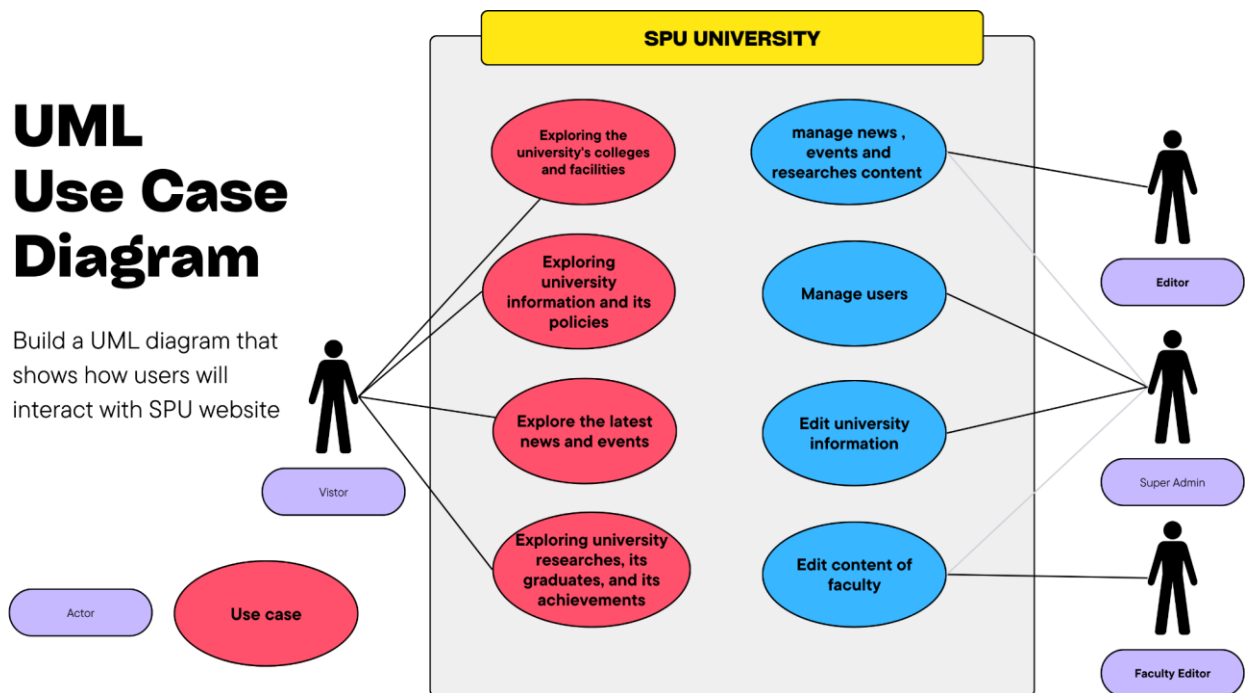


Figure 4.2: Low-Level Visitor Use Case Diagram

Detailed visitor-centered use case view showing browsing, news, events, facilities, research, alumni, achievements, and campus-life interactions.

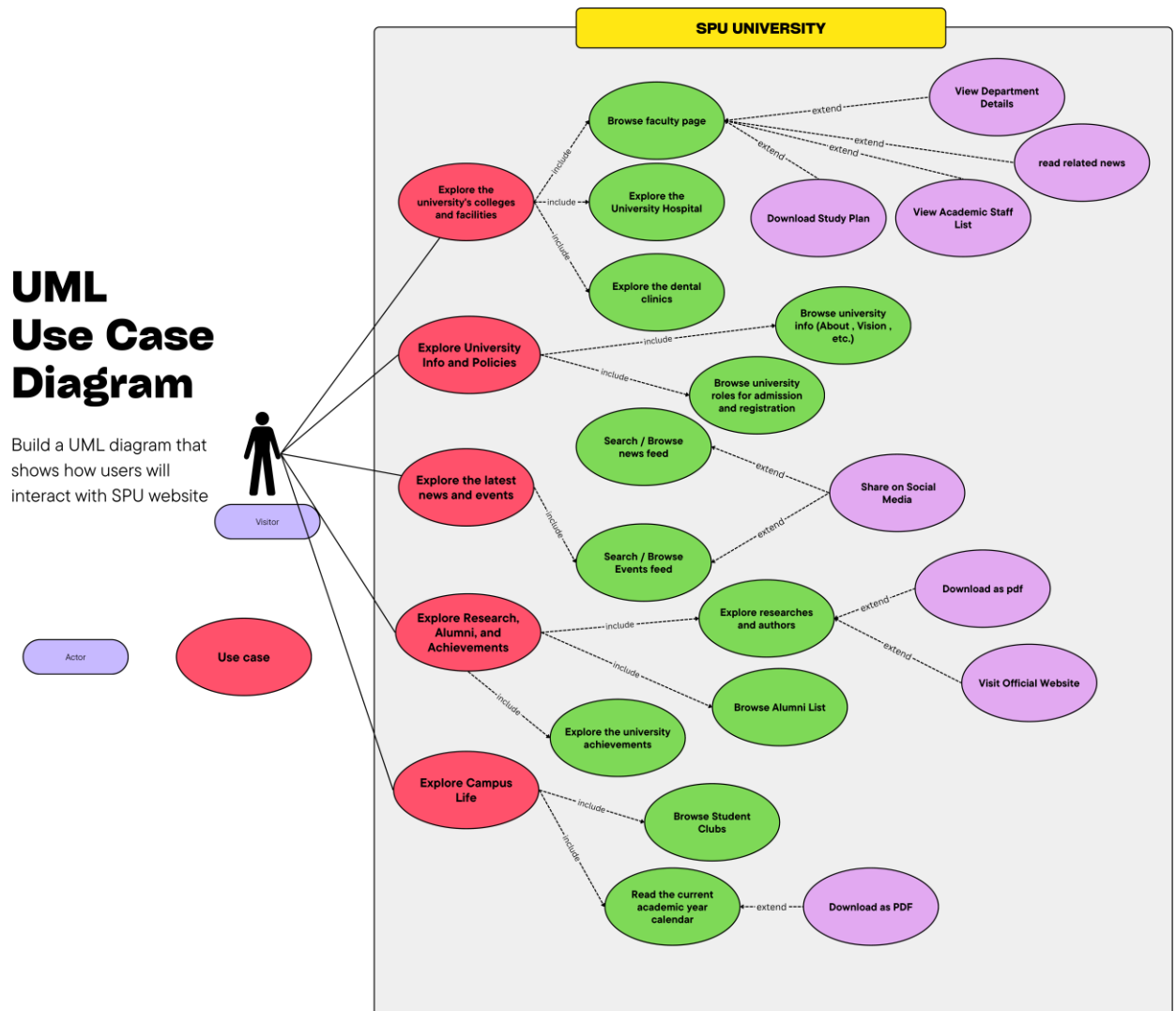


Figure 4.3: Detailed Visitor Use Case Diagram - Team Whiteboard Version

Additional visitor use case view produced during team requirements modeling and discussion.

4.7.2 Use Case Descriptions

Use Case	Actor	Main Flow	Postcondition
Manage Homepage	Editor	Open homepage editor, edit AR/EN section fields, save draft, preview, publish	Published homepage updates and cache is invalidated
Preview Draft	Editor	Open preview URL with token, system validates token, draft renders with preview flag	Draft is visible only to authorized preview flow
Manage Menu	Editor	Open menu builder, edit header/footer/utility items, save changes	Navigation cache is invalidated

Use Case	Actor	Main Flow	Postcondition
Manage Users	Super Admin	Open user resource, edit role, lock/unlock account, save	User permissions are updated and audit logged
View Public Page	Public Visitor	Request locale URL, middleware sets locale, page renders with SEO and navigation	Visitor sees published localized page

4.7.3 Activity Diagrams

The following activity diagrams document the main workflow processes directly within this chapter.

Activity Diagram	Main Steps
AD-01: Apply Form Review Workflow	Student submits apply form -> Staff review -> Validate completeness -> Notify student if incomplete -> Send confirmation if complete
AD-02: Admin Login and MFA Workflow	Admin enters credentials -> Validate -> Lock after repeated failures -> Trigger MFA challenge -> Verify MFA -> Start admin session
AD-03: Faculty Editor Content Approval Workflow	Faculty editor creates/edits content -> Move to review -> Approver reviews -> Return if rejected -> Publish or schedule if approved
AD-04: News Article Editorial Workflow	Create article with AR/EN fields -> Check bilingual completeness -> Send to approver -> Return if rejected -> Publish if approved

Figure 4.4: Accepted Student Apply Form Activity Diagram

Activity flow for reviewing an accepted student application submission and either requesting resubmission or sending confirmation.

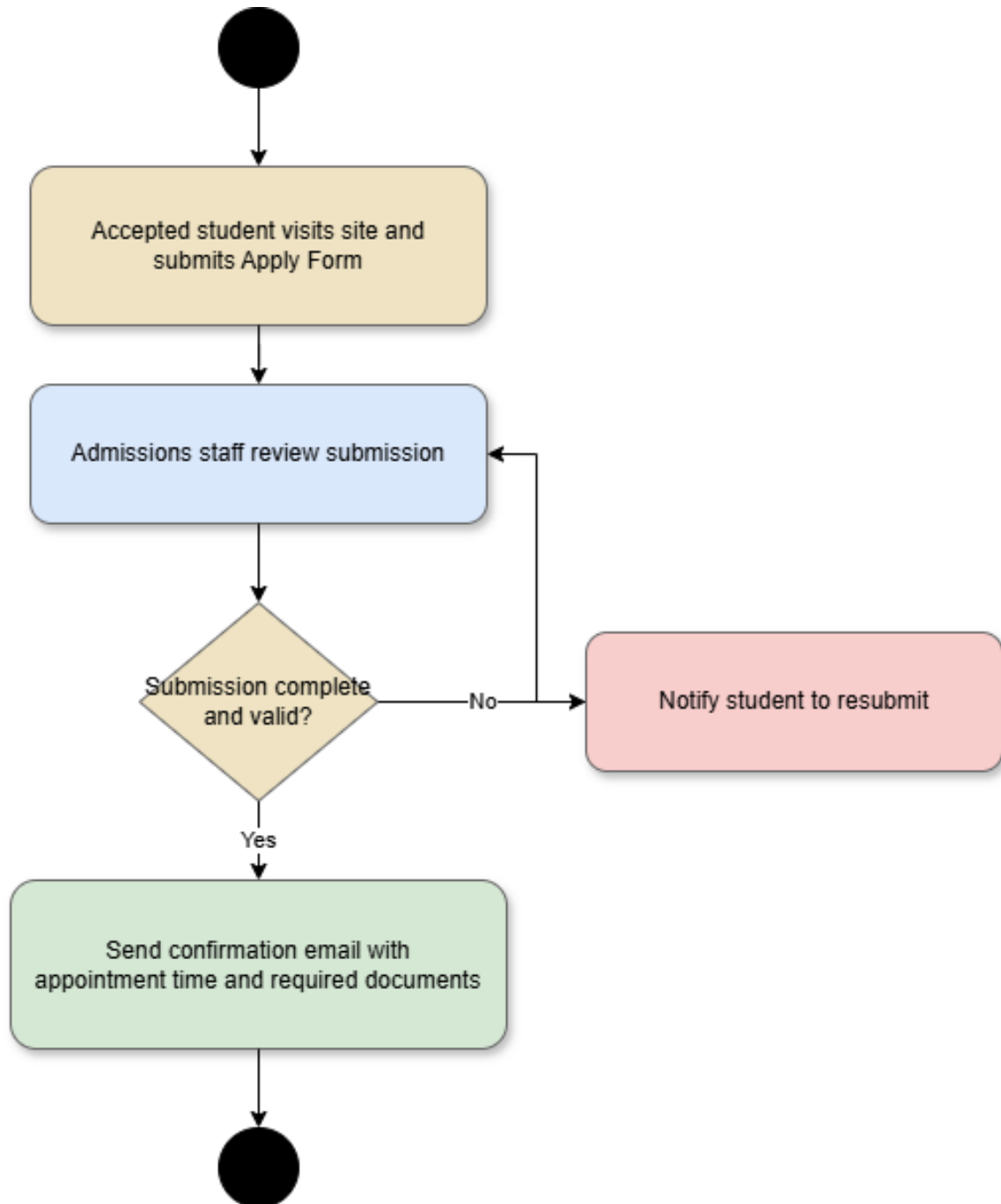


Figure 4.5: Admin Login and MFA Activity Diagram

Activity flow for administrator authentication, credential validation, account lockout, MFA challenge, and session start.

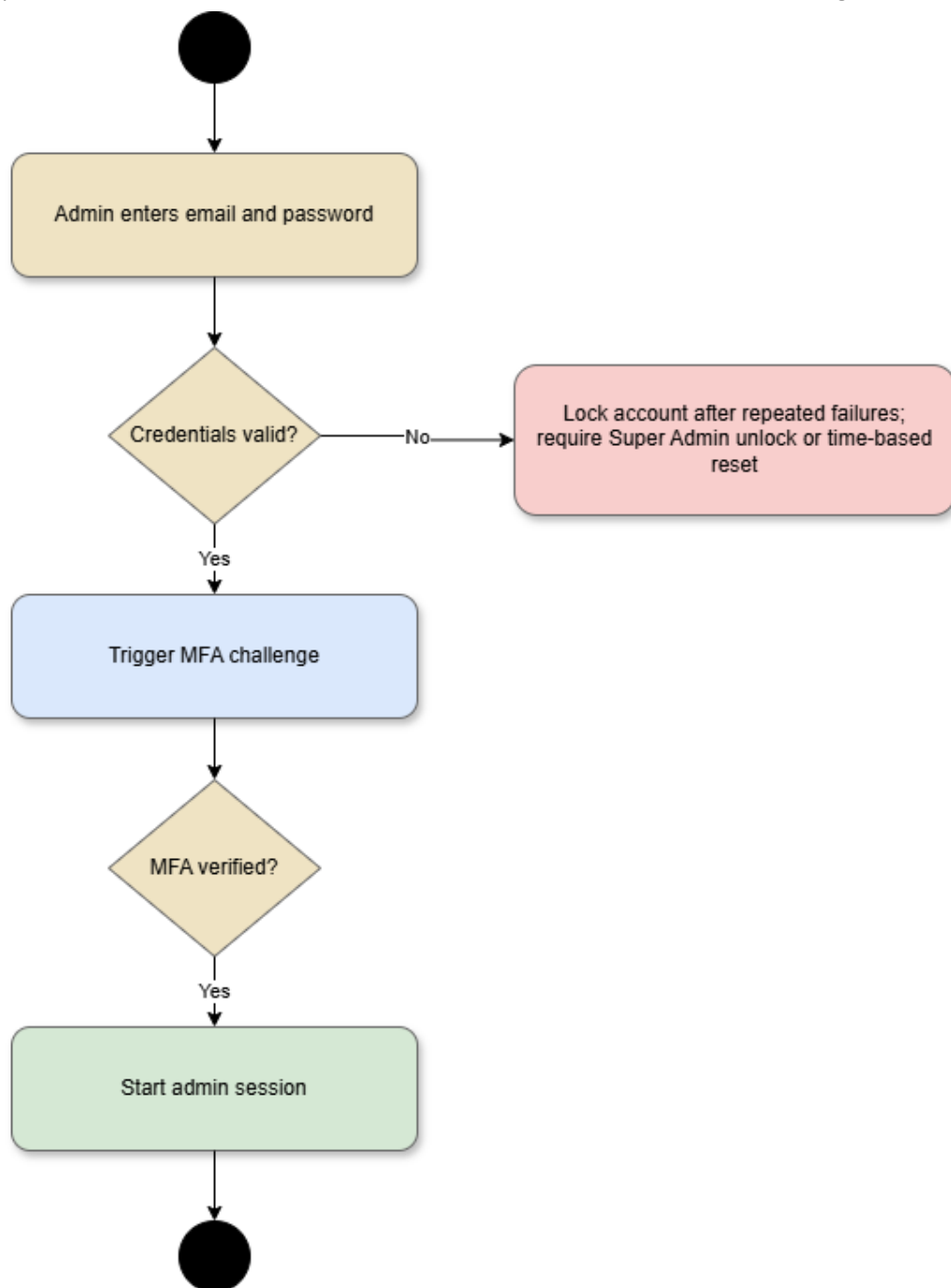


Figure 4.6: CMS Content Publishing Workflow Activity Diagram

Activity flow for faculty editor content creation, review, rejection feedback, and immediate or scheduled publishing.

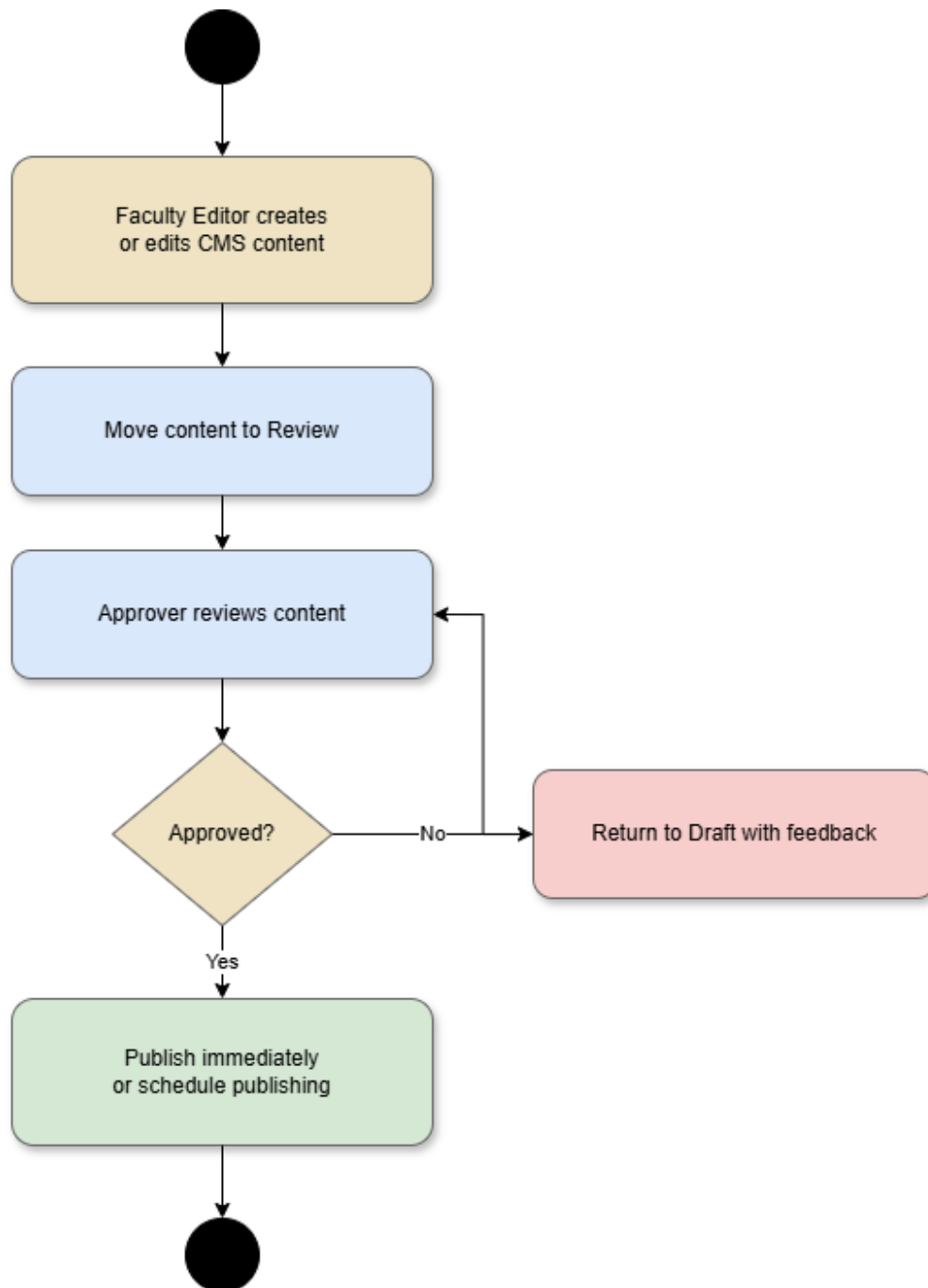


Figure 4.7: News Article Creation and Freshness Activity Diagram

Activity flow for bilingual news article creation, content-completeness validation, approval, feedback, and publication.

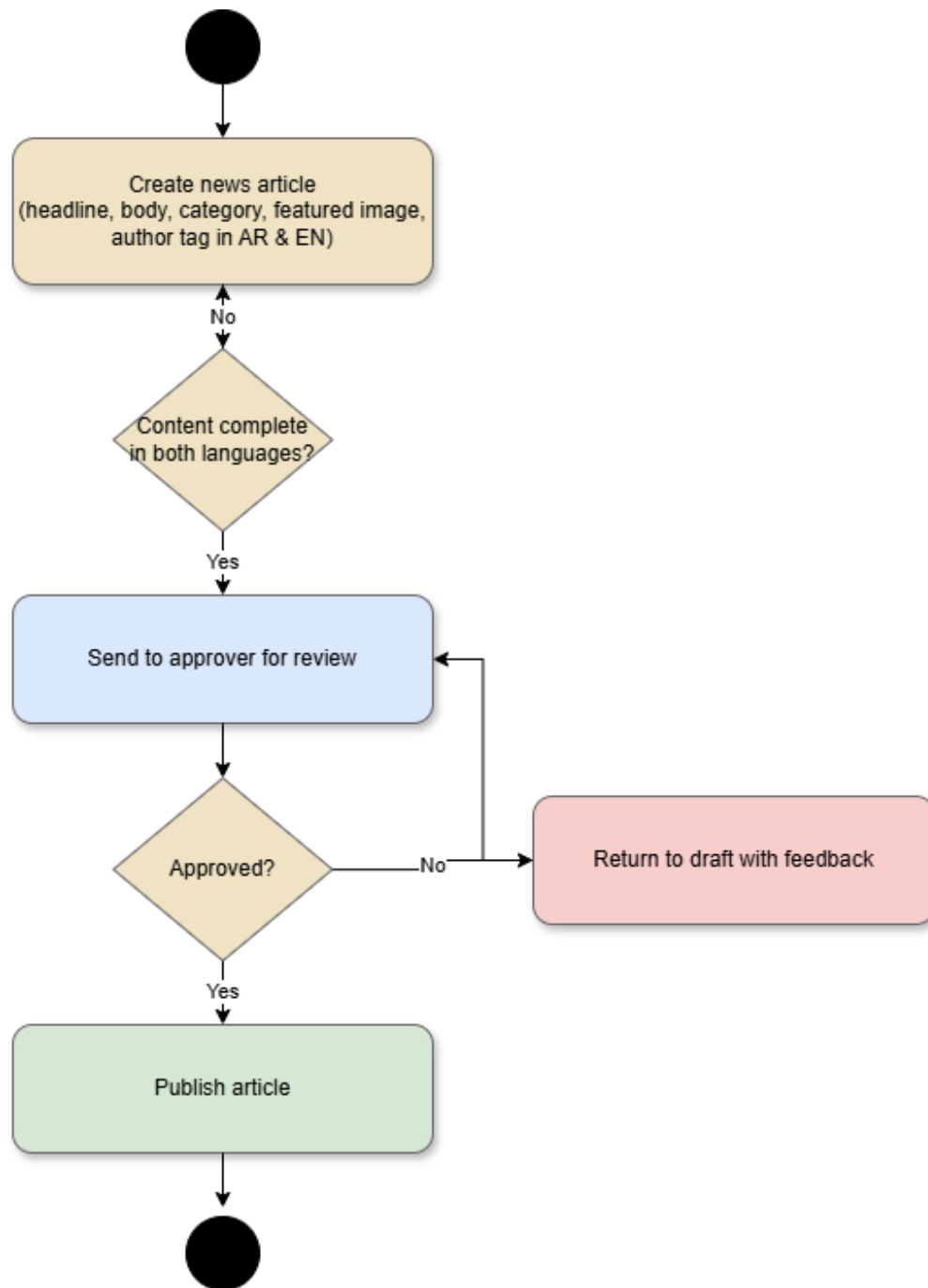
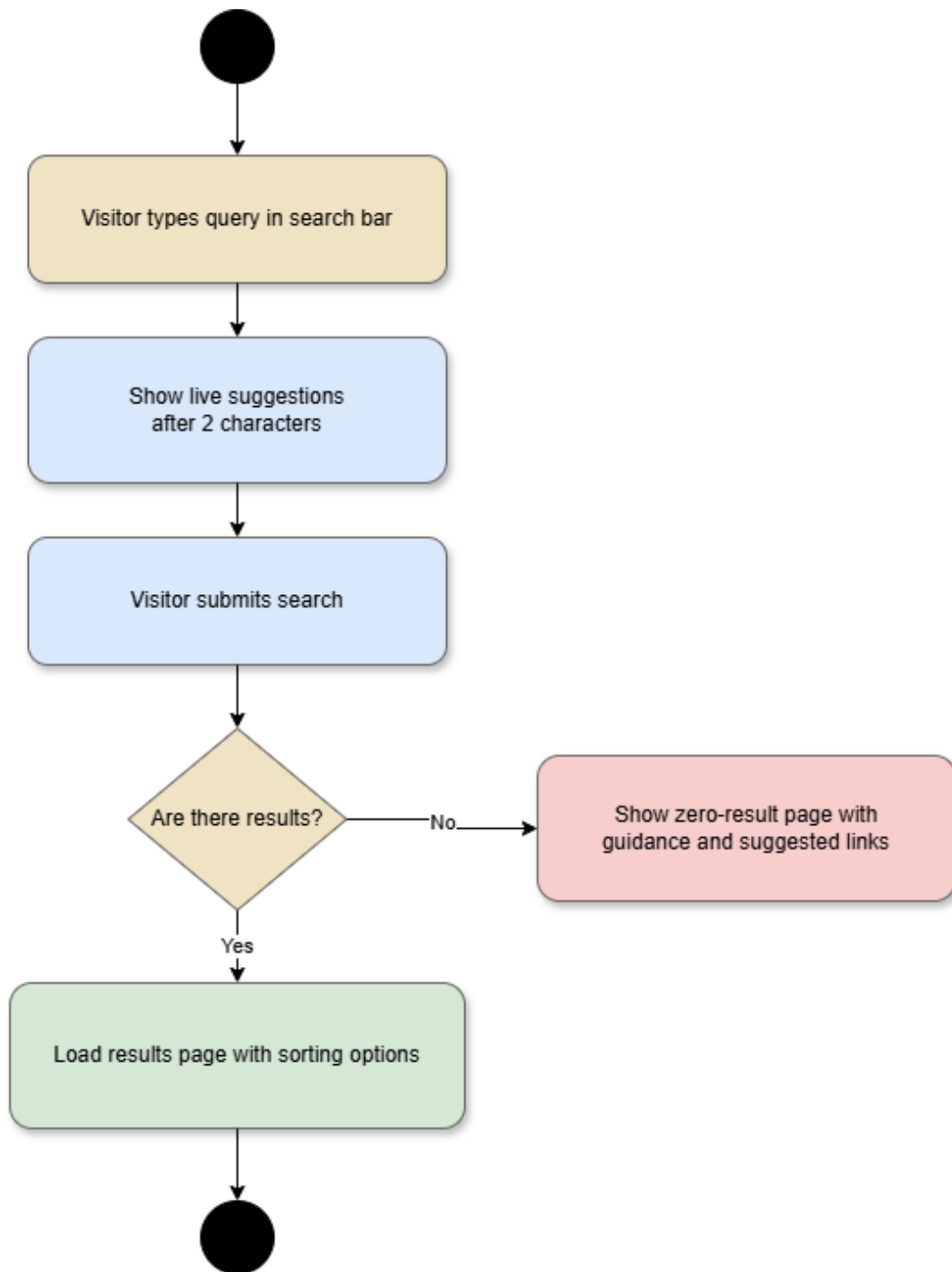


Figure 4.8: Site Search Flow Activity Diagram

Activity flow for site search input, live suggestions, result detection, and zero-result guidance.



4.7.4 Sequence Diagrams

Three sequence diagrams document interaction flows across browser, router, middleware, controller, service layer, model, database, cache, queue, and email participants.

Sequence Diagram	Interaction
SD-01: News Listing Request	Visitor clicks News -> Browser sends request -> Router passes through middleware -> Controller calls service -> Service checks Redis cache -> On miss, queries published news -> Returns result to view
SD-02: Research Page Lookup	Visitor opens research page -> Middleware validates request -> Controller asks service layer -> Service checks cache -> On miss, queries published research data -> Returns result or 404
SD-03: Request Information Form Submission	Prospective student submits form -> Validate CSRF and rate limits -> Validate inputs -> Create lead record -> Queue email -> Return success or validation response

Figure 4.9: News Listing Sequence Diagram

Sequence diagram showing visitor request flow for news listing through browser, router, middleware, controller, service layer, model, database, and Redis cache.

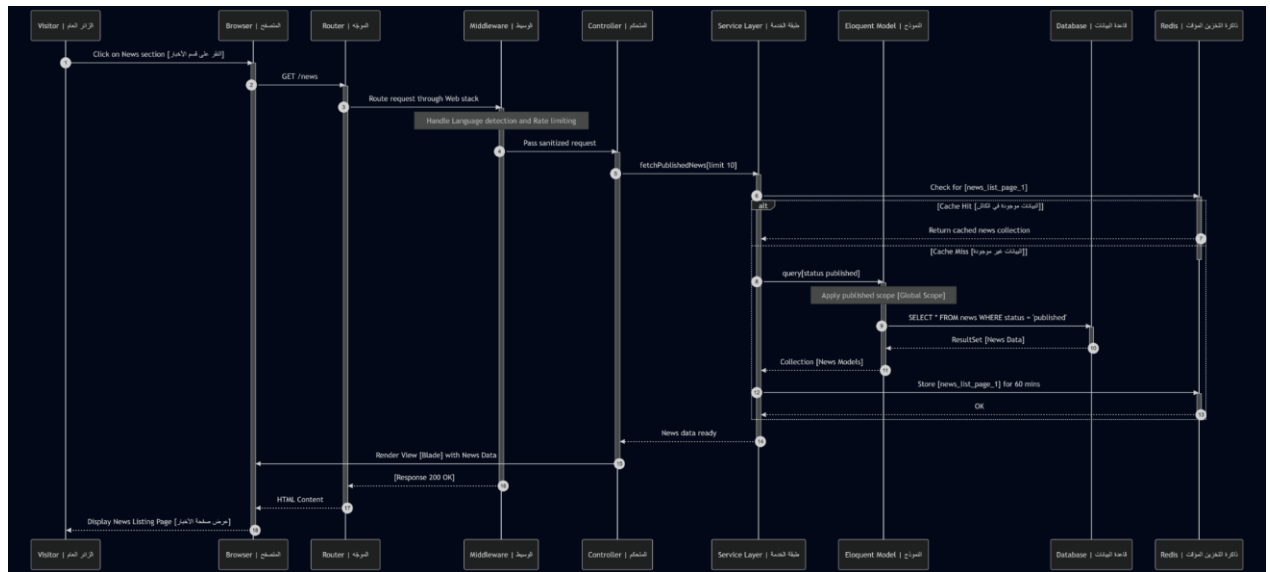


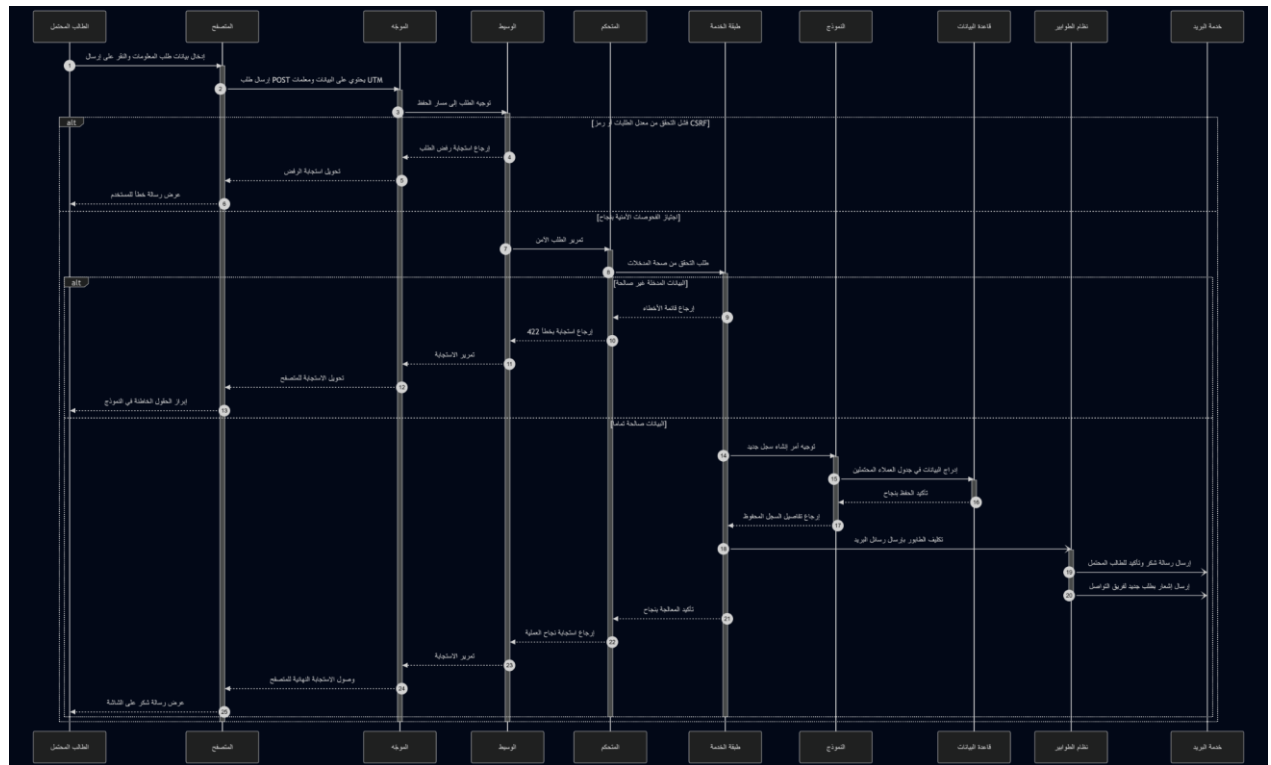
Figure 4.10: Research Page Lookup Sequence Diagram

Sequence diagram showing research-page lookup behavior including cache lookup, database fallback, 404 handling, and optional external search integration.



Figure 4.11: Request Information Form Submission Sequence Diagram

Sequence diagram showing prospective student form submission, CSRF validation, rate limiting, database persistence, and email notification.



4.7.5 Structural and Component Diagrams

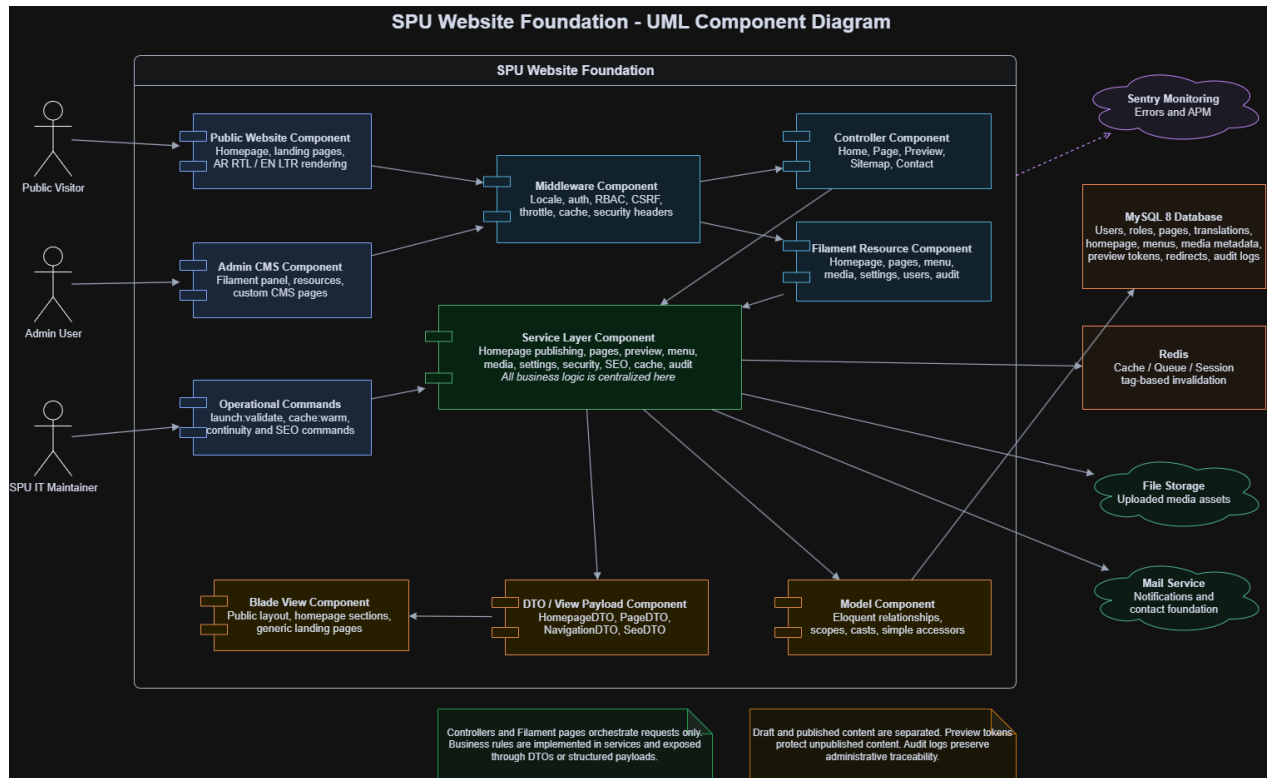
After the behavioral requirements and interaction flows are documented, the structural diagrams present the internal organization of the implemented system. The class diagram models the main entities and relationships, while the component diagram summarizes the technical modules and external dependencies.

Figure 4.12: Class Diagram

Structural class diagram showing the main models, relationships, and service-level responsibilities in the SPU Website Foundation.

Figure 4.13: UML Component Diagram

Component view showing the public website, admin CMS, middleware, controllers, Filament resources, service layer, DTO/view payloads, models, database, Redis, storage, mail, and monitoring integration.



4.8 Requirements Traceability Matrix

Requirement	Goal	Design Component	Test Case	Evidence in Results
SW-FR-001	G-03	Routing	TC-001	Chapter 9 routing result
SW-FR-002	G-03	HomeController, HomepageService	TC-002	Chapter 9 bilingual rendering
SW-FR-003	G-03	Locale middleware, Blade layout	TC-003	Chapter 9 locale behavior
SW-FR-004	G-01, G-02	HomepageSection model/service	TC-004	Chapter 9 CMS section coverage
SW-FR-005	G-04	HomepageDraft, publishing service	TC-005	Chapter 9 draft safety
SW-FR-006	G-04	PreviewToken, PreviewService	TC-006	Chapter 9 preview result
SW-FR-007	G-04	HomepagePublishingService	TC-007	Chapter 9 publish workflow
SW-FR-009	G-02, G-03	PageResource, PageService	TC-008	Chapter 9 page builder result
SW-FR-011	G-05	Policies, Gates, Role model	TC-009	Chapter 9 authorization result
SW-FR-013	G-05	Page/media scoping	TC-010	Chapter 9 faculty scope result
SW-FR-014	G-02, G-03	MenuService, ManageMenu	TC-011	Chapter 9 navigation result
SW-FR-016	G-02	MediaService, MediaAssetResource	TC-013	Chapter 9 media result
SW-FR-019	G-05	AuditService, AuditLog model	TC-014	Chapter 9 audit result
SW-FR-020	G-06	SeoMetadataService	TC-015	Chapter 9 SEO result
SW-FR-021	G-06	SitemapService	TC-016	Chapter 9 sitemap result
SW-FR-023	G-06	ContinuityService	TC-017	Chapter 9 continuity result
SW-NFR-011	G-01	Architecture guard	TC-020	Chapter 9 maintainability result

4.8.1 Sources Supporting Diagram Ordering

Object Management Group. (2017). Unified Modeling Language (UML), Version 2.5.1. OMG Formal Specification. This source supports the use of UML as a language for visualizing, specifying, constructing, and documenting software artifacts.

IEEE Computer Society. (2009). IEEE Std 1016-2009: IEEE Standard for Information Technology—Systems Design—Software Design Descriptions. This source supports organizing design documentation through different views that communicate design concerns to stakeholders.

ISO/IEC/IEEE. (2022). ISO/IEC/IEEE 42010:2022, Software, systems and enterprise—Architecture description. This source supports the use of architecture views and viewpoints to address stakeholder concerns.

4.9 Public Frontend Interfaces

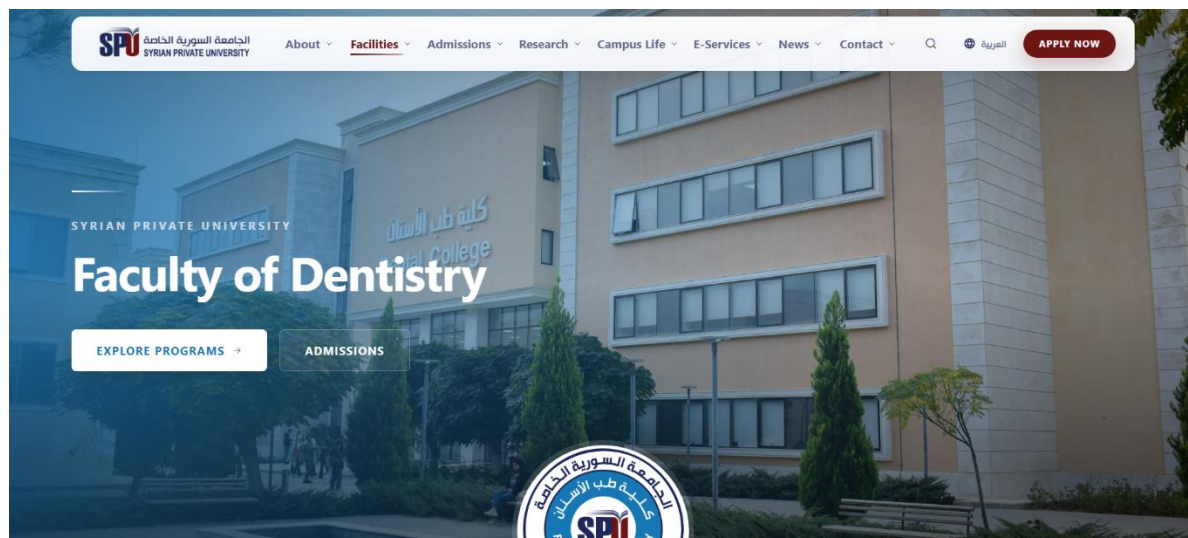


Figure 1Dynamic Faculty Landing Page

This interface dynamically applies the specific faculty's branding, including color themes and logos, based on the URL parameters. It utilizes Alpine.js for seamless client-side tab switching (Overview, Vision, Mission, Objectives) to enhance user experience without full page reloads. The Dean's message component features a responsive glassmorphic overlay, demonstrating a modern UI approach built entirely with Tailwind CSS utility classes

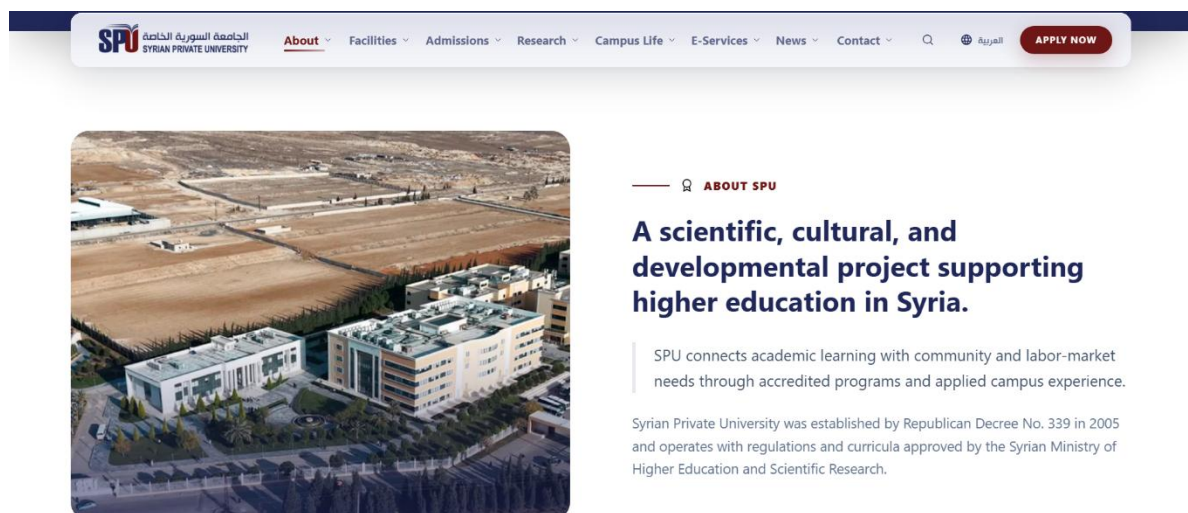


Figure 2University About Page Layout

The "About SPU" page presents the institution's foundational information and history. The layout organizes large textual blocks into readable columns using Tailwind CSS grid utilities. It is designed to meet accessibility standards and fully supports automatic Right-to-Left (RTL) rendering for the Arabic locale, ensuring a consistent bilingual experience.

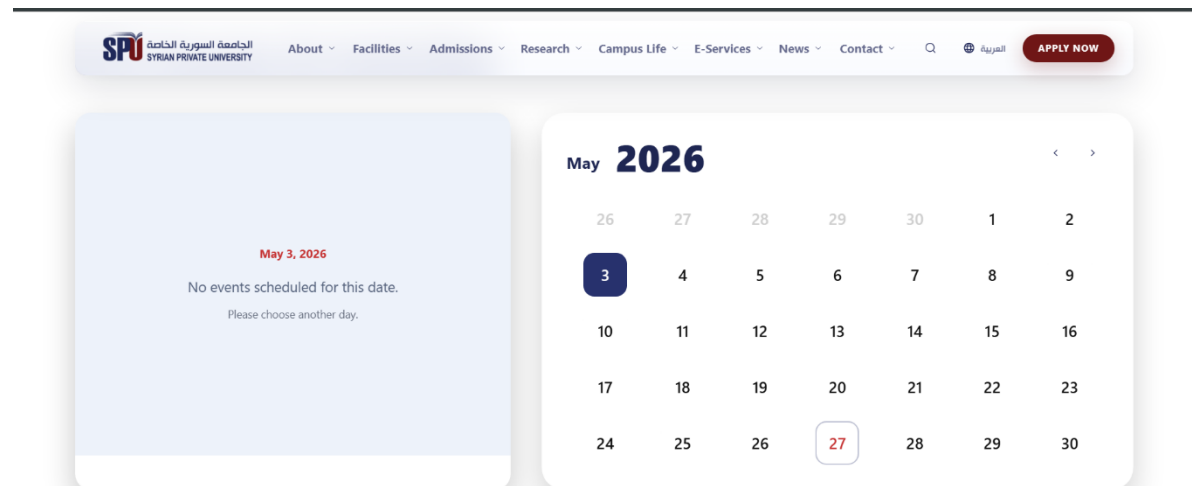


Figure 3 Responsive Academic Calendar Interface

The academic calendar displays official university schedules and deadlines. To ensure usability across all devices, it is built with a responsive table layout that allows horizontal scrolling on smaller viewports. Visual hierarchy is applied using the university's primary brand colors to highlight current semesters and critical events.

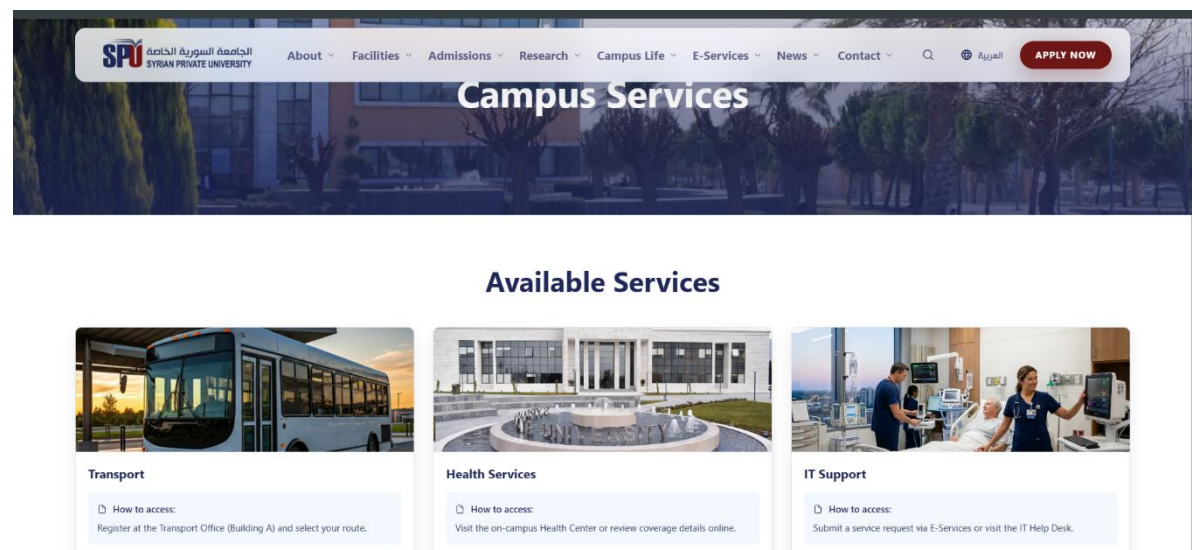


Figure 4 Campus Facilities and Infrastructure Gallery

This section showcases the university's infrastructure, laboratories, and classrooms. The interface employs a modern masonry-style grid system. Smooth interactive hover states and image scaling transitions are achieved exclusively through Tailwind CSS utilities, providing a rich user experience without the overhead of heavy JavaScript libraries.

Figure 5 Contact Us and Interactive Map Interface

The contact interface integrates dynamic inquiry forms, office directories, and geographical map embeds. To ensure data integrity and user feedback, the form implements real-time client-side validation using Alpine.js before securely submitting the payload to the Laravel backend for processing and sanitization.

4.10 Chapter Summary

This chapter defined the requirements for the SPU website foundation. It identified stakeholders, functional requirements, non-functional requirements, constraints, priorities, UML modeling needs, user stories, and traceability links. The requirements confirm that the project is not only a visual redesign but a governed CMS platform with bilingual public delivery, secure administration, publish workflow, SEO continuity, cache behavior, media management, and auditability.

Chapter 5: Architectural and Detailed Design

This chapter explains how the Syrian Private University website system was designed to satisfy the requirements defined in Chapter 4. The design focuses on the implemented foundation scope: public homepage, public navigation shell, generic bilingual landing-page builder, admin CMS foundation, homepage draft/preview/publish workflow, menu builder, settings, media library, audit logging, SEO continuity, caching, and security controls.

5.1 High-Level System Design

The system follows a layered web application design built on Laravel 12 and Filament v3. The public website renders bilingual pages for visitors while the admin panel provides authenticated CMS functions. The main components are:

Component	Responsibility
Public routes	Render homepage, pages, previews, sitemap, robots.txt, and contact submission.
Admin routes and Filament panel	Provide CMS management through protected admin pages and resources.
Middleware	Apply locale, cache, authentication, rate limiting, CSRF protection, webhook verification, and redirect continuity.
Controllers	Orchestrate requests only; they call interfaces and return views or responses.
Services	Contain business logic, publishing rules, cache invalidation, SEO logic, redirect resolution, media handling, and audit logging.
DTOs	Transfer structured data from services to views without exposing raw Eloquent models.
Models	Represent database tables and define relationships, scopes, casts, and lightweight helpers.
Database	Stores users, roles, pages, translations, SEO metadata, homepage sections, drafts, menus, media, preview tokens, redirects, settings, and audit logs.
Redis	Supports production cache, sessions, queues, and cache tags.

5.2 Architectural Pattern

The selected architectural pattern is MVC with a strict service layer. The implemented architecture follows this rule:

```

Request -> Middleware -> Controller -> Service -> Model -> Database ->
Response

                        |
                        v
                    DTO -> View / JSON
    
```

5.2.1 Rationale for MVC + Service Layer

Reason	Explanation
Maintainability	University websites grow over time. A service layer makes future modules easier to add without duplicating logic in controllers.
Testability	Business workflows can be tested through services and feature tests rather than only through UI behavior.
Security	Authorization, validation, publishing, and audit behavior are centralized in predictable layers.
CMS workflow complexity	Draft, preview, publish, schedule, unpublish, cache invalidation, and audit logging require orchestration beyond simple CRUD.
Bilingual content	Arabic and English content mapping is clearer when handled through service DTOs instead of raw model exposure.
Academic compliance	The architecture follows project rules: controllers thin, models without business logic, services injected by interfaces.

5.2.2 Layer Responsibilities

Layer	Responsibilities	Restrictions
Middleware	Locale, authentication, role checks, CSRF, throttling, public cache, redirect continuity, webhook signature validation	No business logic and no direct Eloquent queries.
Controllers	Receive request, call service interfaces, pass DTOs/payloads to views, return response	No Eloquent imports, no direct queries, no business decisions.
Services	Business logic, publishing workflows, mapping models to DTOs, validation, cache invalidation, audit logging	Public methods must not return raw Eloquent models.
DTOs	Structured read-only data transfer between service and view layers	No business logic.
Models	Relationships, scopes, casts, and simple accessors	No business logic and no controller-level usage.
Policies/Gates	Authorization decisions for roles and resources	Must not bypass role checks.

5.3 Architectural Decision Records

ADR-001: Use Laravel 12 as the Backend Framework

Element	Description
Context	The system needs routing, middleware, authentication integration, migrations, services, testing, cache support, and a maintainable PHP ecosystem.
Decision	Use Laravel 12 with PHP 8.2+.
Rationale	Laravel provides strong productivity, mature documentation, built-in testing, database migrations, service container, middleware, queues, cache support, and compatibility with Filament.
Consequences	The team must follow Laravel conventions and ensure business logic stays in services rather than controllers or models.

ADR-002: Use Filament v3 for the Admin CMS

Element	Description
Context	The project requires admin pages for homepage management, menus, settings, pages, media, users, audit logs, and two-factor setup.
Decision	Use Filament v3.
Rationale	Filament is Laravel-native, open-source, productive, and supports resources, custom pages, forms, tables, policies, and role-based admin workflows.
Consequences	Admin UI development is faster, but complex workflows such as homepage publishing still require custom services and custom Filament pages.

ADR-003: MVC with Strict Service Layer

Element	Description
Context	Publishing workflows, SEO logic, cache invalidation, role scoping, and bilingual mapping require orchestration beyond simple CRUD.
Decision	Use MVC with a strict service layer and interface contracts.
Rationale	The pattern is strong enough for the foundation scope and simpler than microservices. It supports testing, separation of concerns, and future extension.
Consequences	More DTOs and interfaces are required, but architecture remains clear and testable.

ADR-004: Explicit Translation Tables

Element	Description
Context	The system must support Arabic and English content independently.

Element	Description
Decision	Use explicit translation tables such as `page_translations`, `homepage_section_translations`, and locale-specific `page_seo_meta`.
Rationale	Explicit tables are clearer, queryable, and safer for structured bilingual content. They also support future content governance.
Consequences	More migrations and models are needed, but content is easier to validate and manage per entity type.

ADR-005: Use MySQL 8 and ADR-006: Use Redis for Production Cache

Element	Description
MySQL Decision	Use MySQL 8 as the main relational database for users, roles, pages, translations, menus, media, settings, redirects, and audit logs.
MySQL Rationale	MySQL is widely supported, familiar in Laravel deployments, stable for relational CMS data, and suitable for university-owned infrastructure.
Redis Decision	Use Redis in production for cache, sessions, and queues, with array cache for local development where needed.
Redis Rationale	Redis supports tag-compatible caching, fast reads, queues, and sessions. Appropriate for public page cache and operational commands such as cache warming.

ADR-007: Protected Preview Tokens

Element	Description
Context	Editors need to preview drafts, but unpublished content must not leak publicly.
Decision	Use tokenized preview URLs with hashed token storage.
Rationale	Preview tokens allow controlled preview without making drafts public. They also support device-specific and locale-specific preview behavior.
Consequences	Preview requests must bypass public cache and validate token expiry and target type.

5.4 Database Design

5.4.1 Core Entities

Entity	Purpose
`users`	Stores admin user accounts, authentication fields, role relation, lock status, and two-factor data.
`roles`	Stores role names such as `super_admin`, `editor`, and `faculty_editor`.

Entity	Purpose
`pages`	Stores generic landing-page records, status, slug, template, schedule, and scope fields.
`page_translations`	Stores Arabic and English page content fields.
`page_seo_meta`	Stores locale-specific SEO metadata for pages.
`page_drafts`	Stores unpublished draft payloads for pages.
`homepage_sections`	Stores the fixed homepage section records and ordering.
`homepage_section_translations`	Stores Arabic and English homepage section content payloads.
`homepage_drafts`	Stores draft homepage payloads before publishing.
`menu_items`	Stores header, footer, and utility navigation items with nesting and locale behavior.
`settings`	Stores global site settings such as footer, contact, emergency notice, social links, and SEO defaults.
`media_assets`	Stores uploaded media metadata, file paths, type, size, dimensions, locale metadata, and faculty scope.
`preview_tokens`	Stores protected preview token metadata and hashed token values.
`audit_logs`	Stores append-only administrative action history.
`legacy_exact_redirects`	Stores exact old URL to new URL redirect rules.
`legacy_pattern_rules`	Stores pattern-based redirect rules with priority.
`unresolved_legacy_requests`	Stores unresolved legacy URL and file requests for follow-up.

5.5 API and Route Design

5.5.1 Public Routes

Method	Route	Handler	Purpose
GET	`/`	Redirect	Redirects to `/ar`, the default Arabic homepage.
GET	`/sitemap.xml`	SitemapController	Returns XML sitemap.
GET	`/robots.txt`	SitemapController	Returns environment-aware robots.txt.
GET	`/{locale}`	HomeController	Renders Arabic or English homepage.
GET	`/{locale}/preview`	PreviewController	Renders token-protected preview content.
POST	`/{locale}/contact`	PublicContactController	Handles throttled public contact form submission.

Method	Route	Handler	Purpose
GET	`/{locale}/{slugPath}`	PageController	Renders published landing pages by slug path.

5.5.2 Admin and Webhook Routes

Method	Route	Purpose
GET	`/admin/login`	Shows admin login page.
POST	`/admin/login`	Handles rate-limited login.
POST	`/admin/auth/logout`	Logs out authenticated admin user.
GET	`/admin/two-factor-challenge`	Shows two-factor challenge.
POST	`/admin/two-factor-challenge`	Verifies TOTP challenge.
POST	`/webhook/incoming`	Accepts HMAC-SHA256 verified incoming webhook.

5.6 Security and Authorization Design

5.6.1 Role Permissions

Role	Permissions
`super_admin`	Full access to homepage, pages, menu, media, settings, users, and audit logs.
`editor`	Access to homepage, pages, menu, media, and settings, but not user management or audit logs.
`faculty_editor`	Scoped access to pages and media matching `faculty_scope_slug`; no homepage, menu, settings, user, or audit access.

5.6.2 Threat Model (STRIDE)

STRIDE Category	Threat Example	Mitigation in Design
Spoofing	Attacker attempts to log in as an admin user	Password authentication, rate limiting, account lockout, TOTP support.
Tampering	Unauthorized user modifies CMS content	Gates, policies, role checks, scoped faculty access, audit logging.
Repudiation	Admin denies publishing or changing content	Append-only audit logs record user, action, entity, timestamp, and metadata.
Information Disclosure	Draft content becomes public or sensitive data leaks	Tokenized preview, public cache bypass, draft exclusion from sitemap.
Denial of Service	Repeated requests overload the system	Throttling, Redis cache, public page caching, cache warming.

STRIDE Category	Threat Example	Mitigation in Design
Elevation of Privilege	Faculty editor accesses unrelated content or user management	Faculty scoping, policies, role-based visibility, architecture tests.

5.7 CMS Workflow Design

The CMS workflow separates draft content from published content. This is essential because university content often requires review before publication.

```
Edit section content -> Save draft -> Generate protected preview -> Review ->
Publish or Schedule -> Write translations -> Invalidate cache -> Create audit
log
```

Workflow Element	Design Decision
Draft save	Allows editors to work without changing the public site.
Preview	Uses protected tokenized URLs and bypasses public cache.
Publish	Validates required payloads, writes published translations, invalidates cache, and records audit log.
Schedule	Allows content to become public at a future time.
Unpublish	Removes content from public visibility.
Optimistic locking	Uses draft version fields to reduce overwrite risk between concurrent editors.

5.8 SEO, Cache, and Continuity Design

Design Area	Implementation Decision
Canonical URLs	Generated as absolute and locale-correct URLs.
Hreflang	Arabic and English alternates are generated where translations exist.
Sitemap	Includes only published and enabled pages; excludes drafts, scheduled, disabled, admin, and preview URLs.
Robots.txt	Environment-aware so staging can be blocked and production can be indexed.
Legacy redirects	Exact redirects have priority over pattern redirects; loop detection is enforced.
Unresolved requests	Unresolved legacy URLs and files are logged for later reconciliation.
Public cache	Locale-aware cache keys; bypass for authenticated users, admin routes, preview flows, and non-GET requests.

Design Area	Implementation Decision
Cache invalidation	Publishing, page updates, menu updates, and settings updates invalidate relevant cache tags.

5.9 DTO and View Payload Design

DTO	Purpose
HomepageDTO	Contains locale, direction, and homepage sections.
HomepageSectionDTO	Represents one homepage section with key, order, enabled state, and localized payload.
HomepageSectionDataDTO	Represents section content: title, summary, image, actions, stats, articles, research items, events.
NavigationPayloadDTO	Contains header, footer, utility trees, language switch links, CTA, portal URLs, emergency notice.
MenuItemDTO	Represents one navigation item with parent, URL, active state, depth, and children.
PageSeoDTO	Represents title, description, OG metadata, canonical URL, hreflang, and robots directive.
PublicSettingsDTO	Represents public settings such as apply CTA, footer, emergency notice, social contact, and default SEO.
PreviewDTO	Represents preview target and token metadata.

5.10 Chapter Summary

The SPU website foundation was designed as a Laravel 12 and Filament v3 system using MVC with a strict service layer. The design separates public rendering, admin CMS workflows, services, DTOs, models, policies, cache, and database responsibilities. It supports bilingual Arabic and English content, RTL/LTR behavior, fixed homepage sections, generic landing pages, tokenized preview, draft/publish workflows, RBAC, media handling, audit logging, SEO metadata, sitemap generation, legacy redirect continuity, and Redis-backed cache behavior.

Chapter 6: Implementation Environment and Tools

This chapter documents the implementation environment, development tools, dependency versions, repository workflow, CI configuration, and reproducibility steps for the Syrian Private University website project. This chapter focuses on the environment and tools only. The internal code structure and implementation details are discussed in Chapter 7.

6.1 Development Environment

The project was developed on Windows 11 machines using a local Laravel development setup. The shared source code repository was hosted on GitHub at <https://github.com/Zainab19hm/Spu-Website>.

The local development workflow used Composer for PHP dependencies, npm for frontend dependencies, Vite for asset bundling, and Laravel Artisan commands for application setup, migrations, serving, testing, and operational tasks.

URL	Purpose
`http://localhost:8000/ar`	Arabic public homepage
`http://localhost:8000/en`	English public homepage
`http://localhost:8000/admin`	Filament admin panel

6.2 Languages, Frameworks, and Libraries

Category	Tool / Technology	Version	Usage	Reason for Selection
Operating System	Windows 11	Dev machine OS	Local development	Available team environment compatible with PHP, Composer, Node.js, and MySQL.
Backend Language	PHP	^8.2	Laravel backend	Required by Laravel 12 and modern typed PHP features.
Backend Framework	Laravel	^12.0	Routing, middleware, services, migrations, Blade, testing	Mature PHP framework with strong ecosystem and service container.
Admin Panel	Filament	3.3	CMS admin panel	Laravel-native admin panel for resources, forms, tables, and custom pages.

Category	Tool / Technology	Version	Usage	Reason for Selection
Database	MySQL	8.x	Main relational database	Suitable for structured CMS data, translations, users, roles, and audit logs.
Cache / Queue / Session	Redis	Production	Cache, queues, sessions	Supports fast cache and tag-based invalidation.
Frontend Build Tool	Vite	^7.0.7	Asset bundling	Modern Laravel-supported frontend build system.
CSS Framework	Tailwind CSS	^4.0.0	Responsive styling	Utility-first CSS framework suitable for responsive UI implementation.
JavaScript	Alpine.js	^3.14.9	Lightweight interactivity	Small frontend library for dropdowns, toggles, and simple interactions.
Icons	FontAwesome SVG	^7.2.0	Public and admin icons	Provides consistent icon set.
HTML Sanitization	HTMLPurifier	^4.19	Sanitizing CMS HTML	Reduces stored XSS risk in rich text content.
Two-Factor Authentication	Google2FA Laravel	^3.0	TOTP-based MFA	Adds multi-factor authentication support for admin accounts.
Monitoring	Sentry Laravel	^4.25	Error tracking and APM	Provides production error visibility and performance monitoring.
Testing	PHPUnit	^11.5.50	Automated tests	Laravel-compatible feature and unit testing.
Static Analysis	Larastan / PHPStan	^3.0	Static analysis	Helps identify type and code quality issues.
Formatting	Laravel Pint	^1.24	PHP style checking	Maintains code style consistency.

6.3 Git and Branch Strategy

Branch Type	Purpose
`main`	Stable version of the project.
`develop`	Active integration branch for ongoing work.
`feature/*`	Feature-specific development work such as homepage CMS, media library, or SEO.

Branch Type	Purpose
`hotfix/*`	Urgent fixes for critical issues.

6.4 CI/CD Pipeline

The repository contains a GitHub Actions workflow named Security CI. It runs on pull requests and pushes to `main` or `master`. The CI workflow uses Ubuntu as the runner environment, PHP 8.2, and Node.js 22.

CI Step	Purpose
Checkout	Fetches repository source code.
Setup PHP	Installs PHP 8.2 with required extensions such as mbstring, dom, fileinfo, mysql, and redis.
Setup Node	Installs Node.js 22 and enables npm caching.
Install PHP dependencies	Runs Composer installation.
Install Node dependencies	Runs npm dependency installation.
Prepare environment	Copies `.env.example` and generates `APP_KEY`.
Composer validate	Validates Composer configuration.
Composer audit	Checks PHP dependencies for known vulnerabilities.
npm audit	Checks frontend dependencies for high-severity vulnerabilities.
Pint	Checks Laravel PHP code formatting.
PHPStan	Runs static analysis through PHPStan/Larastan.
Tests	Runs the Laravel test suite using `php artisan test`.

6.5 Containers and Infrastructure

Docker was not used as the primary development or deployment mechanism in this project phase. The implemented workflow relies on a local Windows 11 setup with PHP, Composer, Node.js, npm, MySQL, and Laravel Artisan commands.

Infrastructure Item	Production Requirement
Application environment	`APP_ENV=production` and `APP_DEBUG=false`
Canonical URL	HTTPS origin such as `https://spu.edu.sy`
Database	MySQL database with least-privileged application user, not `root`

Infrastructure Item	Production Requirement
Cache	Redis unless another tag-compatible backend is approved
Session driver	Redis or approved production session backend
Queue driver	Redis or another production queue backend with workers configured
Session cookies	Secure, HTTP-only, SameSite lax or stricter
Secrets	Stored outside git and not copied from local development
Monitoring	Sentry DSN configured for production error tracking and APM

6.6 Reproducibility

6.6.1 Local Setup Steps

The standard setup commands are:

```
git clone https://github.com/Zainab19hm/Spu-Website.git
cd Spu-Website
composer install
cp .env.example .env
php artisan key:generate
npm install
npm run build
php artisan migrate:fresh --seed
php artisan serve
```

URL	Expected Result
`http://localhost:8000/ar`	Arabic homepage
`http://localhost:8000/en`	English homepage
`http://localhost:8000/admin`	Admin login page

6.6.2 Composer Setup and Development Scripts

The project defines Composer scripts for automation:

```
composer run setup # Install dependencies, create .env, generate key, run
migrations, build assets
composer run dev # Start Laravel server, queue listener, log viewer, and
Vite dev server
composer run test # Run the test suite
```

6.6.3 Testing Commands

```
php artisan test
php artisan test --exclude-group=property
```

```
php artisan test --filter=ArchitectureGuardTest
php artisan test --group=property
```

6.7 Runtime Environments

Environment	Description	Main Configuration
Local	Developer machine for implementation and testing	Windows 11, PHP, Composer, Node.js, npm, MySQL, local `.env`, optional array cache
Staging	Production-like environment for validation before launch	Production-style configuration, noindex behavior, launch validation, test data
Production	Live SPU website environment	HTTPS, MySQL, Redis, secure sessions, Sentry, strong secrets, launch checklist, backup and rollback plan

6.8 Operational Commands

Command	Purpose
`launch:validate`	Runs pre-launch checks for homepage rendering, SEO, sitemap, redirects, cache, and audit behavior.
`cache:warm`	Warms homepage, navigation, settings, and sitemap caches.
`continuity:validate-redirects`	Detects duplicate, conflicting, or looping redirect rules.
`continuity:export-url-inventory`	Exports redirect rules as JSON or CSV.
`continuity:export-file-inventory`	Exports legacy file continuity inventory.
`continuity:report-unresolved`	Reports unresolved legacy requests.
`continuity:validate-seo`	Identifies pages with missing or weak SEO metadata.
`continuity:reconciliation-report`	Produces a combined continuity and SEO report.

6.9 Chapter Summary

The SPU website was developed on Windows 11 using a Laravel 12, PHP 8.2, MySQL 8, Redis, Filament v3, Tailwind CSS, Alpine.js, Vite, PHPUnit, and GitHub-based workflow. The repository is hosted at <https://github.com/Zainab19hm/Spu-Website>. The project includes clear local setup commands, Composer scripts, npm scripts, CI security checks, production environment templates, launch validation commands, and rollback documentation.

Chapter 7: Software Implementation

This chapter explains how the architectural design from Chapter 5 was implemented in the Syrian Private University website project. The purpose of this chapter is not to reproduce the entire source code, but to show representative implementation decisions and explain how the main modules work.

7.1 Project Structure

SPU_Website/	
app/	
Contracts/	Service interfaces used for dependency injection
DTOs/	Read-only data transfer objects
Services/	Business logic and workflow orchestration
Models/	Eloquent models for database tables
Http/	
Controllers/	Thin public and admin controllers
Middleware/	Locale, cache, auth, headers, webhook, and
security middleware	
Filament/	
Pages/	Custom admin pages such as ManageHomepage,
ManageMenu, ManageSettings	
Resources/	Admin resources for pages, media, users, and audit
logs	
Policies/	Authorization policies
Console/Commands/	Launch, cache, continuity, and SEO commands
Providers/	Container bindings and application bootstrapping
database/	
migrations/	Database schema migrations
seeders/	Baseline seed data
resources/views/	Blade public and admin views
routes/web.php	Public, admin, preview, sitemap, robots, and
webhook routes	
tests/	
Feature/	Feature and workflow tests
Unit/	Unit and service tests

7.2 Public Controllers

7.2.1 Homepage Controller

HomeController receives the locale from the route, loads the public homepage through HomepageSectionServiceInterface, loads navigation through NavigationServiceInterface, loads settings through SettingsServiceInterface, and prepares SEO data.

```
public function __invoke(Request $request, string $locale): View
{
    $homepage = $this->homepageSectionService->getPublicHomepage($locale);
```

```

    abort_if(! $this->hasRenderableHomepage($homepage), 404);

    $navigation = $this->navigationService->getFullNavigationPayload($locale,
$request->path());
    $homeShell = $this->pageService->getPublicPageBySlug('home', $locale);

    return view('public.home', [
        'locale' => $locale,
        'direction' => $homepage->direction,
        'homepage' => $homepage,
        'navigation' => $navigation,
        'settings' => $this->settingsService->getPublicSettings($locale),
        'languageSwitch' => $navigation->languageSwitchLinks,
        'isPreview' => false,
    ]);
}

```

7.2.2 Page Controller

PageController renders generic landing pages. It asks PageServiceInterface for the published page by slug and locale, rejects missing or homepage-shell pages, builds navigation, selects the correct translation and SEO DTO, and passes a flattened payload to the public page view. The controller converts the PageDTO and PageTranslationDTO into a view payload supporting the generic landing-page builder without requiring every future section to have its own controller.

7.3 Service Layer Implementation

7.3.1 Homepage Publishing Service

HomepagePublishingService implements the homepage draft, publish, schedule, and unpublish workflow. It uses authorization gates, optimistic locking, transactions, payload normalization, HTML sanitization, cache invalidation, and audit logging.

Method	Responsibility
saveDraft()	Saves a draft snapshot with versioning and audit log entry.
publish()	Validates draft, writes homepage sections and translations, marks draft as published, invalidates cache, and logs the action.
schedulePublish()	Marks a valid draft as scheduled for a future publish time.
unpublish()	Disables public homepage sections and invalidates cache.

```

DB::transaction(function () use ($draft, $sections, $userId): void {
    foreach ($sections as $section) {
        $sectionModel = HomepageSection::query()->updateOrCreate(
            ['key' => $section->key],

```

```

        [
            'type' => $this->sectionType($section->key),
            'sort_order' => $section->sortOrder,
            'is_enabled' => $section->isEnabled,
            'schema_version' => 1,
        ],
    );

    HomepageSectionTranslation::query()->updateOrCreate(
        ['section_id' => (int) $sectionModel->getKey(), 'locale' => 'ar'],
        ['payload_json' => $this->sanitizeSectionPayload(...)],
    );
}

$draft->forceFill(['status' => 'published', 'published_at' => now()])->save();
});

```

7.3.2 Page Service

PageService handles generic page creation, metadata updates, translations, SEO metadata, draft lifecycle, publishing, and breadcrumbs. It delegates public reads to PagePublicReadService, draft payload conversion to PageDraftService, and URL resolution to PageUrlResolver. The service supports independent Arabic and English translation updates through updateArabicTranslation() and updateEnglishTranslation() methods.

7.3.3 Preview Service

PreviewService creates and validates preview tokens, resolves preview payloads, and builds preview DTOs. Token lifecycle operations are delegated to PreviewTokenStore. This prevents draft content from being accessible through normal public URLs while still allowing authorized reviewers to inspect unpublished work.

7.3.4 Menu Service

MenuService implements the menu builder logic for header, footer, and utility navigation. It supports page, route, and URL targets; sanitizes labels and URLs; enforces a maximum depth of two levels; invalidates navigation caches; and records audit logs.

```

$parent = $this->parentForPayload($payload->parentId, $payload->groupKey, $locale);
$depth = $parent instanceof MenuItem ? ((int) $parent->depth + 1) : 0;

$this->assertDepthAllowed($depth);
$this->assertTargetConfiguration($payload, $locale);

```

7.3.5 Media Service

MediaService implements upload, metadata editing, listing, and soft deletion for media assets. It validates file type, file size, image dimensions, authorization, and faculty scope.

Constraint	Value
Maximum file size	20 MB
Maximum image width	8000 px
Maximum image height	8000 px
Supported types (examples)	JPEG, PNG, GIF, WebP, PDF, Office documents, MP4, WebM

7.3.6 Audit Service

AuditService records administrative actions in the audit_logs table. Each log includes action name, user ID, entity type, entity ID, metadata, IP address, and user agent. Audit logging is called by services such as homepage publishing, page management, menu management, and media management.

7.4 Admin Panel Implementation

Admin Page / Resource	Purpose
ManageHomepage	Custom page for editing the fixed 11-section homepage.
ManageMenu	Custom page for managing header, footer, and utility navigation.
ManageSettings	Custom page for global settings: footer, contact, social links, emergency notice, and SEO defaults.
PageResource	CRUD and workflow management for generic landing pages.
MediaAssetResource	Uploading, editing, viewing, filtering, and soft-deleting media.
UserResource	Super-admin-only user role and lock management.
AuditLogResource	Read-only audit log viewer.
TwoFactorSetup	Authenticated page for configuring TOTP-based two-factor authentication.

7.5 Data Access Layer

Model	Purpose
User	Admin users, roles, lock status, two-factor fields.
Role	Role names and permissions basis.
Page	Generic landing-page records.

Model	Purpose
PageTranslation	AR/EN page content.
PageSeoMeta	Locale-specific SEO metadata.
PageDraft	Draft payloads for pages.
HomepageSection	Fixed homepage section records.
HomepageSectionTranslation	AR/EN homepage section payloads.
HomepageDraft	Draft homepage snapshots.
MenuItem	Header, footer, and utility navigation tree items.
MediaAsset	Uploaded media metadata and file paths.
Setting	Global settings.
PreviewToken	Protected preview token metadata.
AuditLog	Administrative action history.
LegacyExactRedirect / LegacyPatternRule / UnresolvedLegacyRequest / LegacyFileInventory	Migration and continuity support.

7.6 Business Logic Layer

Business Rule	Implementation Location
Draft content must not affect public content until published	HomepagePublishingService, PageService, PreviewService
Draft preview must be token-protected	PreviewService, PreviewTokenStore
Homepage sections must use approved keys	HomepageSectionService, HomepagePublishingService
Menu nesting cannot exceed two levels	MenuService
Admin actions must be authorized by role	Policies, Gates, services, Filament pages
Faculty editors are scoped to their own faculty content	Policies and service checks
Public cache must be invalidated after content changes	CacheService calls from publishing, page, menu, and settings services
Admin write operations must be auditable	AuditService calls across services
CMS HTML must be sanitized	HtmlSanitizer and service-level sanitization

7.7 User Interface Implementation

The public interface uses Blade templates with Tailwind CSS and Alpine.js. The layout file defines the HTML language and direction attributes based on the active locale:

```
<html lang="{{ $locale }}" dir="{{ $direction }}">
```

This implements RTL for Arabic and LTR for English. The homepage view is driven entirely by CMS section data:

```
@foreach ($homepage->sections as $section)
    @continue($section->key === 'footer')
    @continue(! $section->isEnabled)
    @include('public.partials.homepage-section', ['section' => $section,
'locale' => $locale])
@endforeach
```

Navigation uses Alpine.js for dropdowns and mobile behavior. Language switching is generated from LanguageSwitchLinkDTO values rather than hardcoded links.

7.8 Error Handling and Logs

Error Case	Handling
Public homepage has no renderable hero section	HomeController returns 404 through `abort_if`.
Public page is missing, draft, disabled, or homepage shell	PageController returns 404.
User lacks permission	Service or Filament page raises authorization exception through Gate/Policy.
Draft version conflict	Service logs conflict, dispatches event, and throws ConflictException.
Invalid uploaded file	MediaService throws validation exception.
Missing preview token	PreviewService returns null or fails validation.

7.9 Security Implementation

Security Control	Implementation
Admin authentication	Custom admin login routes and Filament admin panel.
Rate limiting	`admin-login`, `two-factor`, and `public-form` rate limiters.
MFA	Google2FA/TOTP challenge and setup flow.

Security Control	Implementation
Account lockout	Lock after repeated failed attempts.
RBAC	Gates and policies for users, pages, media, menu, settings, homepage, and audit logs.
Faculty scoping	`faculty_scope_slug` on pages and media.
CSRF protection	Laravel CSRF middleware for browser forms.
Webhook security	HMAC-SHA256 signature verification middleware.
HTML sanitization	HTMLPurifier allowlist for CMS content.
Upload validation	MIME type, extension, size, and image dimension checks.
Security headers	Middleware adds protective headers where configured.
Audit logs	Database-backed log for admin actions.

7.10 Implementation Summary

The SPU website implementation follows the architecture defined in Chapter 5. Controllers remain thin, services contain business logic, DTOs isolate views from raw models, Filament implements the admin CMS, Blade renders server-side public pages, and middleware protects locale, cache, authentication, throttling, and security behavior. The most important implemented features form a stable foundation for future full-site modules.

Chapter 8: Testing and Evaluation

This chapter presents the testing strategy, test types, test cases, results, quality measurements, and evaluation evidence for the Syrian Private University website project. The testing approach follows the intent of ISO/IEC/IEEE 29119-3:2021 by documenting the test plan, test cases, expected results, actual results, and acceptance decision.

8.1 Test Plan

8.1.1 Test Scope

Area	Included in Testing
Public website	Homepage rendering, landing pages, locale headers, language switching, preview flow, 404 behavior.
Admin panel	Login, logout, access control, Filament resources, homepage management, pages, menu, settings, media, users, audit logs.
CMS workflow	Draft save, preview token, publish, schedule, unpublish, cache invalidation, audit logging.
Security	RBAC, account lockout, TOTP challenge, input sanitization, upload validation, webhook signature verification.
SEO	Canonical URLs, hreflang, meta tags, sitemap, robots.txt, SEO fallback behavior.
Continuity	Exact redirects, pattern redirects, file continuity, unresolved request logging, loop detection.
Architecture	Controllers do not import models, contracts resolve, no legacy kernel, homepage keys match architecture.
Operations	Launch validation, cache warming, export/report commands, rollback-related checks.

8.1.2 Entry and Exit Criteria

Criterion Type	Criteria
Entry criteria	Code builds locally, Composer dependencies installed, npm dependencies installed, test database available, <code>`.env.testing`</code> configured, migrations runnable.
Exit criteria	All required tests pass, no critical failures remain, public draft content is not exposed, admin authorization works, launch validation checks are available, and documented limitations are known.

8.1.3 Test Environment

Testing was performed in the Laravel local test environment on Windows 11. PHPUnit 11 was used through Laravel's ``php artisan test`` command. The non-property test suite was executed with:

```
php artisan test --exclude-group=property
```

The executed test result was: Tests: 395 passed (1169 assertions) | Duration: 320.88s

8.2 Types of Testing

Test Type	Purpose	Tools / Evidence
Unit testing	Verify isolated services and helpers such as slug generation, media upload validation, TOTP, HTML sanitization, and form schema generation.	PHPUnit
Feature testing	Verify end-to-end Laravel features such as public runtime, admin authorization, homepage workflow, middleware, and Filament resources.	Laravel Feature Tests
Integration testing	Verify service interaction with database records, translations, settings, page publishing, and public reads.	PHPUnit + SQLite/MySQL-compatible migrations
Architecture testing	Verify architecture constraints: no Eloquent imports in controllers and all contracts having bindings.	ArchitectureGuardTest
Security testing	Verify RBAC, account logout, TOTP challenge, webhook signatures, sanitization, upload validation, and faculty scoping.	PHPUnit tests + CI audits
SEO testing	Verify canonical URLs, hreflang, robots.txt, sitemap, and SEO fallback behavior.	Feature tests and validation commands
Cache testing	Verify cache headers, cache bypass, cache warm command, and invalidation after writes.	Middleware tests and command tests
Manual launch checklist	Verify staging/production readiness items not fully covered by automated tests.	Launch readiness checklist

8.3 Test Cases

Test Case	Requirement	Expected Result	Actual Result	Status
TC-001	SW-FR-001	Redirects to `/ar`	Verified by routing/runtime tests	Passed

Test Case	Requirement	Expected Result	Actual Result	Status
TC-002	SW-FR-002	Both homepages render by locale	Public runtime tests passed	Passed
TC-003	SW-FR-003	Arabic uses RTL, English uses LTR	Locale/header tests passed	Passed
TC-004	SW-FR-004	Fixed 11 homepage section keys returned in order	Homepage workflow test passed	Passed
TC-005	SW-FR-005	Public homepage unchanged after draft save	Draft leakage test passed	Passed
TC-006	SW-FR-006	Draft renders only through token	Preview tests passed	Passed
TC-007	SW-FR-007	Public homepage updates, cache invalidates, audit row created	Publish workflow test passed	Passed
TC-008	SW-FR-009	Generic page renders through page builder payload	Page/public runtime tests passed	Passed
TC-009	SW-FR-011	Each role sees only allowed resources	Role visibility tests passed	Passed
TC-010	SW-FR-013	Access denied or hidden for out-of-scope content	Security scoping tests passed	Passed
TC-011	SW-FR-014	Header/footer/utility trees resolve by locale	Navigation shell tests passed	Passed
TC-012	SW-FR-015	Depth greater than two is rejected	Menu depth test passed	Passed
TC-013	SW-FR-016	Valid files accepted; invalid MIME/SVG/oversized rejected	Media service tests passed	Passed
TC-014	SW-FR-019	Audit log created	Audit tests passed	Passed
TC-015	SW-FR-020	Meta, canonical, hreflang, robots render correctly	SEO rendering tests passed	Passed
TC-016	SW-FR-021	Sitemap includes only published enabled pages	Sitemap tests passed	Passed
TC-017	SW-FR-023	Exact/pattern redirects resolve; loops terminate	Continuity tests passed	Passed
TC-018	SW-NFR-009	Script/event handlers removed	Sanitizer tests passed	Passed

Test Case	Requirement	Expected Result	Actual Result	Status
TC-019	SW-NFR-018	Valid code passes; invalid code rejected	Two-factor tests passed	Passed
TC-020	SW-NFR-011	Controllers do not import models; contracts resolve	Architecture guard passed	Passed
TC-021	SW-NFR-002	Cache header changes to HIT; preview bypasses cache	Middleware cache tests passed	Passed

8.4 Test Results and Defect Analysis

Test Group	Result
Total tests executed	395
Total assertions	1,169
Failed tests	0
Error tests	0
Skipped tests	0
Duration	320.88 seconds

Earlier testing phases documented several defects that were corrected during implementation:

Defect	Resolution
`ContinuityService::logUnresolved()` used a non-existing raw expression method	Replaced with a valid expression implementation.
Some migrations had SQLite compatibility issues	Adjusted prefix indexes and duplicate index names for test compatibility.
Example test lacked database refresh isolation	Added the required refresh database behavior.

8.5 Quality Metrics

Metric	Target	Measured / Verified Result	Status
Automated non-property tests	All pass	395/395 passed	Met
Assertions	No failed assertions	1,169 assertions passed	Met
Architecture compliance	Controllers avoid direct model imports; contracts bind correctly	ArchitectureGuardTest passed	Met

Metric	Target	Measured / Verified Result	Status
Draft safety	Draft content must not leak publicly	Homepage and public runtime tests passed	Met
RBAC	Roles must be restricted correctly	Admin authorization and role visibility tests passed	Met
Faculty scoping	Faculty editor cannot access out-of-scope content	Security scoping regression tests passed	Met
SEO correctness	Canonical, hreflang, robots, sitemap behavior verified	SEO and sitemap tests passed	Met
Cache behavior	Cache keys, bypass, invalidation, cache warm verified	Middleware and cache warm tests passed	Met
Sanitization	Unsafe HTML removed	HtmlSanitizer and PageService sanitization tests passed	Met
Media validation	Invalid types and oversized files rejected	MediaService tests passed	Met
MFA	TOTP and recovery code flows verified	TotpAuthenticator and TwoFactorChallenge tests passed	Met

8.6 Usability Testing

Formal usability testing with 5-8 real participants and SUS scoring was not completed as part of the current automated evidence. However, the system includes implementation and tests that support usability requirements indirectly:

Usability Area	Evidence
Bilingual navigation	Navigation shell tests verify AR/EN payloads.
Mobile navigation structure	Blade layout implements responsive desktop/mobile navigation panels.
Preview banner	Preview banner tests verify preview state visibility.
Accessibility landmarks	Tests verify layout landmarks, headings, article elements, and HTML root attributes.
Menu depth	Navigation tests reject menu depth greater than two.

For final production validation, a SUS questionnaire with 5-8 users is recommended. Suggested tasks include finding admissions information, switching language, browsing a faculty page, opening news/research cards, and using the admin homepage draft/preview workflow.

8.7 AI/ML Model Testing

The implemented project does not include an AI or machine learning model. Therefore, model accuracy, precision, recall, F1-score, bias testing, model cards, dataset datasheets, and drift monitoring are not applicable to the current scope. If future versions add AI-assisted search, chatbot support, recommendation, or content classification, those features must be tested separately using independent datasets and AI governance documentation.

8.8 Security Testing

Security Area	Evidence
Authentication	Admin login/logout tests passed.
Account lockout	Auth service and admin flow tests passed.
MFA	TOTP and challenge tests passed.
RBAC	Admin authorization and role visibility tests passed.
Faculty scoping	Security scoping regression tests passed.
Webhook verification	Valid, missing, invalid, wrong-secret, unconfigured-secret, empty-body, and replayed nonce tests passed.
HTML sanitization	XSS-oriented sanitizer tests passed.
Upload validation	Invalid MIME, SVG bypass attempt, oversized files, and missing file tests passed.
Dependency security	CI workflow includes `composer audit` and `npm audit --audit-level=high`.

8.9 Acceptance Decision

Based on the current automated evidence, the SPU website foundation satisfies the core implemented requirements for the graduation project scope. The executed non-property test suite passed completely with 395 tests and 1,169 assertions. The system demonstrates verified behavior for public rendering, bilingual layout, admin authorization, CMS workflow, preview safety, SEO, sitemap, cache behavior, redirect continuity, media validation, audit logging, MFA, and architecture compliance.

The acceptance decision is: Accepted for graduation-project foundation scope, with the condition that production launch should additionally include performance testing, Lighthouse audits, manual accessibility review, and external security scanning.

Chapter 9: Results and Analysis

9.1 Introduction

This chapter presents the results obtained from implementing and testing the Syrian Private University Website foundation. The analysis focuses on the project objectives defined in Chapter 1, the functional and non-functional requirements modeled in Chapter 4, the architecture designed in Chapter 5, the implementation described in Chapter 7, and the verification results from Chapter 8.

9.2 Summary of Implemented Results

The implemented system delivered a working Laravel-based website foundation for Syrian Private University.

Area	Result
Public homepage	Implemented as an 11-section CMS-driven homepage with Arabic and English content support.
Bilingual routing	Implemented through locale-aware public routes for `/ar` and `/en`.
Direction support	Arabic renders using RTL direction; English renders using LTR direction.
Landing pages	Implemented through a generic page builder with structured blocks.
Admin panel	Implemented using Filament v3 resources and custom pages.
CMS workflow	Draft, preview, publish, schedule, unpublish, and optimistic locking behavior implemented.
Preview safety	Preview access is tokenized and bypasses public cache.
Roles and permissions	Implemented for `super_admin`, `editor`, and `faculty_editor`.
Media library	Implemented with upload validation and scoped access behavior.
Audit logging	Implemented as an append-only record of administrative write operations.
SEO continuity	Implemented metadata, canonical URLs, hreflang alternates, sitemap, robots.txt, redirects, and unresolved request logging.
Caching	Implemented locale-aware public page caching with bypass and invalidation rules.
Automated tests	Current non-property test suite passed with 395 tests and 1,169 assertions.

9.3 Objective Achievement Analysis

Objective	Description	Evidence	Result
G-01	Build a maintainable public website foundation using MVC, strict service layer, and DTO-based transfer.	Controllers are thin, business logic is placed in services, architecture guard tests passed.	Achieved
G-02	Provide an admin panel for managing homepage content, pages, menus, settings, media, users, and audit logs.	Filament admin pages and resources implemented for all foundation CMS operations.	Achieved within foundation scope
G-03	Support Arabic and English content with RTL/LTR behavior and locale-aware routing.	Public routes support `/ar` and `/en`; layout outputs language and direction attributes.	Achieved
G-04	Implement draft, protected preview, schedule, publish, and unpublish workflow without draft leakage.	Homepage and page workflow services implement draft and publish states; preview is tokenized.	Achieved
G-05	Implement security controls including RBAC, logout, two-factor authentication, sanitization, security headers, and audit logging.	Role tests, logout tests, TOTP tests, HTML sanitizer tests, and audit logging behavior verified.	Achieved
G-06	Preserve discoverability and migration continuity through SEO metadata, sitemap, robots, redirects, and unresolved request logging.	SEO rendering tests, sitemap tests, redirect continuity tests, canonical/hreflang behavior implemented.	Achieved

9.4 Functional Results

9.4.1 Public Website Results

The public website provides a bilingual homepage available through Arabic and English locale routes. The homepage is not hardcoded as static HTML content; it is rendered from CMS-managed sections. This directly improves maintainability because content editors can update homepage content through the admin panel rather than requiring manual source-code changes.

Section Key	Result
`hero`	Supported through CMS-managed hero content.
`hero_stats`	Supported through structured statistics content.
`academic_faculties`	Supported through CMS section content.

Section Key	Result
`achievements_highlights`	Supported through CMS section content.
`choose_your_path`	Supported through CMS section content.
`university_news`	Supported as a foundation section for news-style homepage content.
`research_studies`	Supported as a foundation section for research-style homepage content.
`events_activities`	Supported as a foundation section for event-style homepage content.
`medical_facilities_services`	Supported as a foundation section for medical facilities content.
`bottom_stats`	Supported through CMS-managed statistics.
`footer`	Supported through CMS-managed footer content and links.

9.4.2 Admin Panel Results

The administrative panel was implemented using Filament v3 and provides a practical CMS foundation for authorized university staff. The access model separates responsibilities between roles: a `super\_admin` can access all administrative areas, an `editor` can manage content-oriented areas, and a `faculty\_editor` is scoped to permitted faculty-related pages and media.

9.4.3 CMS Workflow Results

The project implemented a complete foundation-level publishing workflow. Editors can save draft content, preview unpublished changes through protected preview URLs, schedule future publishing, publish validated content, and unpublish content when needed. The key result is that draft content does not become publicly visible unless it is explicitly published.

9.4.4 SEO and Continuity Results

The system includes SEO and migration-continuity features: page metadata, canonical URLs, hreflang alternates, XML sitemap generation, robots.txt, legacy redirects, redirect loop protection, and unresolved request logging. This result helps preserve discoverability and reduces the risk of broken links after migration from an older website structure.

9.5 Non-Functional Results

Quality Attribute	Result	Evidence
Maintainability	Improved by MVC separation, services, contracts, DTOs, and thin controllers.	Architecture rules and architecture guard tests.
Testability	Supported through unit, feature, integration, middleware, security, SEO, and workflow tests.	395 non-property tests passed.

Quality Attribute	Result	Evidence
Security	Implemented through RBAC, account lockout, TOTP, sanitization, upload validation, headers, and audit logging.	Security-related automated tests passed.
Localization	Supported through Arabic/English routes, locale-aware rendering, and RTL/LTR layout behavior.	Public runtime and layout tests passed.
Performance foundation	Supported through response caching, cache bypass, cache invalidation, and cache warm command.	Cache middleware and cache warm tests passed.
SEO continuity	Supported through metadata, sitemap, robots, redirects, and unresolved request logging.	SEO, sitemap, and continuity tests passed.
Governance	Supported through roles, policies, audit logs, and controlled publishing workflow.	Admin authorization and audit workflow tests passed.

9.6 Automated Testing Results

Metric	Result
Total tests executed	395
Total assertions	1,169
Failed tests	0
Error tests	0
Duration	320.88 seconds

9.7 Research Question Answers

9.7.1 RQ-01: Maintainability and Governance

The result shows that the custom Laravel-based CMS foundation significantly improves maintainability and governance. Content is managed through an admin panel instead of manual code edits. Business logic is isolated in services, controllers remain thin, dependencies are resolved through interfaces, and DTOs are used for structured data transfer. Governance is improved through role-based permissions, protected publishing workflow, optimistic locking, and audit logs.

9.7.2 RQ-02: Bilingual Arabic and English Support

The system effectively supports Arabic and English at the foundation level. Public routes are locale-aware, the homepage and landing pages support Arabic and English content, and the public layout handles correct document language and direction. Arabic uses RTL rendering, while English uses LTR

rendering. This result is important for Syrian Private University because Arabic is the default institutional language while English remains necessary for international visitors.

9.7.3 RQ-03: Draft Safety and Preview Workflow

The result is positive. The implemented workflow separates draft and published states, provides tokenized preview URLs, bypasses public cache for preview flows, and validates publishing operations. Automated tests confirmed that draft content does not leak publicly and that preview behavior is protected.

9.7.4 RQ-04: Quality Attributes

The implemented architecture satisfies the essential quality attributes for the foundation phase. Security controls include RBAC, account lockout, two-factor authentication, sanitization, upload validation, security headers, and audit logs. Testability is demonstrated through a passing automated test suite. Some production-level quality targets such as final Lighthouse scores, full WCAG 2.1 AA manual audit, and load testing require production-like deployment and final content.

9.8 Comparison With the Previous Situation

Aspect	Static or Manual Approach	Implemented CMS Foundation
Content updates	Require developer or direct file changes.	Managed through admin panel.
Publishing safety	Draft and live content can be difficult to separate.	Draft, preview, publish, schedule, and unpublish states are separated.
Bilingual content	Often duplicated manually.	Locale-aware Arabic and English structures are supported.
RTL/LTR behavior	Can be inconsistent if manually handled.	Layout supports language and direction attributes.
Governance	Limited role separation.	RBAC and policies control admin access.
Accountability	Hard to trace content changes.	Audit logging records administrative writes.
SEO continuity	Redirects and metadata may be incomplete.	Sitemap, robots, metadata, canonical, hreflang, and redirect continuity are supported.
Maintainability	Changes can become scattered.	Service layer, contracts, DTOs, and tests improve structure.

9.9 Limitations of the Results

Although the foundation objectives were achieved, several limitations remain. First, the project intentionally focused on the homepage and CMS foundation. Full modules were not fully

implemented. Second, final performance, accessibility, and security measurements should be collected in a production-like environment. Third, formal usability testing with real students, staff, and administrators was not completed. Fourth, the CMS foundation supports managed content, but final production value depends on complete, accurate, and approved Arabic and English content entered by the university.

9.10 Discussion

The results demonstrate that the project succeeded as an engineering foundation. The most important achievement is not only the visual homepage, but the controlled system behind it: services, contracts, DTOs, permissions, preview protection, publishing workflow, audit logging, and tests. An official university website is an operational system that must be maintainable, secure, bilingual, and governed.

Overall, the results support the main hypothesis introduced in Chapter 1. A purpose-built CMS foundation using Laravel, Filament, DTOs, service-layer architecture, role-based authorization, and automated tests can provide a more maintainable and institutionally controlled website foundation than a static website or an unstructured generic implementation.

9.11 Conclusion

The implemented Syrian Private University Website foundation achieved its planned objectives. It provides a bilingual public website, CMS-managed homepage, admin panel, protected publishing workflow, security controls, SEO continuity, cache behavior, and automated verification. The project remains correctly limited to the homepage and admin foundation scope, while leaving full domain-specific modules for future work.

Chapter 10: Conclusion and Recommendations

10.1 Introduction

This chapter concludes the thesis by summarizing the Syrian Private University Website project, reviewing the achieved objectives, presenting the final conclusions, and proposing recommendations for future development and production deployment. The project was designed and implemented as a bilingual website and CMS foundation for Syrian Private University, with emphasis on maintainability, content governance, Arabic/English support, safe publishing workflow, security, SEO continuity, and automated testing.

10.2 Project Summary

The Syrian Private University Website project addressed the need for a maintainable and institutionally controlled web platform. The project implemented a Laravel-based CMS foundation using MVC architecture with a strict service layer. The system uses Filament v3 for the admin panel, MySQL for persistent data, Redis-oriented caching rules for production, DTOs for structured data transfer, and automated tests to verify critical behavior.

The public side provides Arabic and English locale routes, RTL/LTR rendering behavior, an 11-section CMS-driven homepage, SEO metadata, canonical URLs, hreflang alternates, sitemap generation, robots.txt, and continuity support. The admin side provides controlled management of homepage content, pages, menus, settings, media assets, users, audit logs, and two-factor setup.

10.3 Main Achievements

Area	Achievement
Architecture	Implemented MVC with strict service-layer business logic and interface-based dependency injection.
Public website	Implemented a bilingual public website foundation with Arabic and English routes.
Homepage	Implemented the approved 11-section CMS-managed homepage structure.
Admin panel	Implemented Filament-based administration for foundation CMS operations.
Publishing workflow	Implemented draft, protected preview, schedule, publish, unpublish, and optimistic locking behavior.
Security	Implemented role-based access control, account lockout, two-factor authentication, HTML sanitization, security headers, upload validation, and audit logging.

Area	Achievement
SEO and continuity	Implemented metadata, canonical URLs, hreflang alternates, sitemap, robots.txt, redirect continuity, and unresolved request logging.
Testing	Verified the current non-property suite with 395 passing tests and 1,169 assertions.

10.4 Objective Review

Objective	Final Status
G-01: Build a maintainable public website foundation using MVC, services, and DTOs.	Achieved.
G-02: Provide an admin panel for homepage, pages, menus, settings, media, users, and audit logs.	Achieved within foundation scope.
G-03: Support Arabic and English content with RTL/LTR behavior and locale-aware routing.	Achieved.
G-04: Implement safe draft, preview, schedule, publish, and unpublish workflow.	Achieved.
G-05: Implement security controls including RBAC, lockout, two-factor authentication, sanitization, headers, and audit logs.	Achieved.
G-06: Preserve discoverability and continuity through SEO metadata, sitemap, robots, redirects, and logging.	Achieved.

10.5 Final Conclusions

The first conclusion is that a purpose-built CMS foundation is more suitable for an official university website than a static or manually managed website. Static websites can be simple to deploy, but they become difficult to maintain when content changes frequently, multiple languages are required, and publishing responsibilities must be shared across administrative users.

The second conclusion is that strict architecture rules improved the maintainability of the implementation. By placing business logic in services, using interfaces for dependencies, keeping controllers thin, and using DTOs for data transfer, the project created a codebase that is easier to test, review, and extend.

The third conclusion is that bilingual support must be treated as a core architectural requirement, not as a final visual adjustment. Arabic is the default institutional language while English is important for academic and international communication.

The fourth conclusion is that safe publishing workflow is critical for institutional content governance. Draft separation, protected preview, scheduling, publishing validation, unpublishing, cache bypass, and audit logs reduce the risk of accidental public exposure of incomplete or unapproved content.

The fifth conclusion is that automated testing is necessary for confidence in a CMS foundation. The passing test suite verifies public rendering, admin permissions, CMS workflow, preview safety, SEO, sitemap, cache behavior, media validation, two-factor authentication, security behavior, and architecture compliance.

10.6 Recommendations for Production Deployment

1. Complete final Arabic and English content entry for the homepage, landing pages, menus, metadata, and footer.
2. Review all content with authorized university stakeholders before publication.
3. Run Lighthouse audits for performance, accessibility, best practices, and SEO after final content and hosting are configured.
4. Run an OWASP ZAP or equivalent external security scan against a staging deployment.
5. Run load testing using a tool such as k6, Locust, or JMeter to validate expected visitor traffic.
6. Perform manual WCAG 2.1 AA accessibility review, especially for Arabic RTL pages, keyboard navigation, contrast, form labels, and screen-reader structure.
7. Conduct usability testing with students, faculty members, and administrative editors.
8. Configure production backups, monitoring, alerting, log retention, and incident response procedures.
9. Verify final redirect inventory from the previous website to preserve existing links and search-engine ranking.
10. Prepare editor training material for homepage updates, page editing, media uploads, menu changes, preview review, and publishing approval.

10.7 Recommendations for Maintenance

11. Keep Laravel, PHP dependencies, npm packages, and Filament updated according to a planned maintenance schedule.
12. Run automated tests before every production release.
13. Review audit logs regularly for unusual administrative activity.

14. Monitor unresolved legacy requests to identify missing redirects or frequently requested old URLs.
15. Review SEO metadata and sitemap output after major content changes.
16. Keep Arabic and English content synchronized through editorial review.
17. Enforce role separation and remove inactive admin accounts.
18. Test backup restoration periodically, not only backup creation.
19. Document every production change, especially permission, route, redirect, and publishing workflow changes.
20. Continue applying the service-layer architecture rules when adding future modules.

10.8 Future Work

21. Full admissions module for application information, requirements, deadlines, and applicant workflows.
22. Full news module with categories, archive pages, featured news, and editorial workflow.
23. Full events module with calendars, registration links, event archive, and reminders.
24. Full research repository with publications, author profiles, departments, filtering, and citation metadata.
25. Full faculties and programs module with structured academic departments, plans, staff profiles, and program pages.
26. Contact and CRM module for inquiries, lead tracking, routing, and response history.
27. Advanced search using Meilisearch or another search engine after sufficient content volume exists.
28. Multistage editorial approval workflow for sensitive institutional content.
29. Analytics dashboard for public traffic, content performance, search terms, and broken links.
30. API layer for selected public content if mobile applications or external integrations are planned.

10.9 Lessons Learned

First, defining the scope clearly was essential. A university website can easily expand into many large systems. Limiting the graduation project to the homepage and CMS foundation made the work achievable while still valuable.

Second, bilingual and RTL/LTR support must be planned early. Adding Arabic and English support after implementation would have caused duplicated work and inconsistent behavior.

Third, a CMS requires more than CRUD screens. The important parts are workflow, validation, preview safety, permissions, audit logs, and cache invalidation.

Fourth, automated testing helped protect the architecture from regression. The architecture guard tests and workflow tests were especially useful because they verified not only individual functions, but also project rules.

Fifth, documentation improved traceability. Requirements, architecture rules, implementation reports, launch checklists, and thesis chapters made it easier to connect project goals with actual evidence.

10.10 Closing Statement

The Syrian Private University Website project successfully delivered a bilingual Laravel-based CMS foundation for the university's public web presence. The system supports controlled content management, safe publishing, role-based administration, SEO continuity, caching behavior, security controls, audit logging, and automated testing.

The project demonstrates that a carefully scoped and well-architected CMS foundation can improve maintainability, governance, and readiness for future expansion. With final production validation, real institutional content, and continued development of domain-specific modules, this foundation can serve as the basis for a modern official website for Syrian Private University.

References

This chapter consolidates the thesis references in one final section. Only sources dated 2018 or later are included, as requested, and the list was expanded to cover the standards, UML modeling sources, security guidance, SEO guidance, Laravel/Filament stack, testing tools, database, caching, and monitoring technologies used in the project.

- Ahmad, T., Iqbal, J., Ashraf, A., Truscan, D., & Porres, I. (2019). Model-based testing using UML activity diagrams: A systematic mapping study. *Computer Science Review*, 33, 98-112.
<https://doi.org/10.1016/j.cosrev.2019.07.001>
- Alyami, A., Pileggi, S. F., Sohaib, O., & Hawryszkiewicz, I. (2023). Seamless transformation from use case to sequence diagrams. *PeerJ Computer Science*, 9, e1444. <https://doi.org/10.7717/peerj-cs.1444>
- Filament PHP. (2026). Filament documentation. <https://filamentphp.com/docs>
- Google. (2020). Web Vitals. <https://web.dev/vitals>
- Google Search Central. (2021). Mobile-first indexing best practices.
<https://developers.google.com/search/mobile-sites/mobile-first-indexing>
- Google Search Central. (2025). How to specify a canonical URL with rel="canonical" and other methods.
<https://developers.google.com/search/docs/crawling-indexing/consolidate-duplicate-urls>
- Google Search Central. (2025). Introduction to robots.txt.
<https://developers.google.com/search/docs/crawling-indexing/robots/intro>
- International Organization for Standardization. (2018). ISO 9241-11:2018: Ergonomics of human-system interaction - Part 11: Usability: Definitions and concepts.
<https://www.iso.org/standard/63500.html>
- International Organization for Standardization. (2018). ISO/IEC/IEEE 29148:2018: Systems and software engineering - Life cycle processes - Requirements engineering.
<https://www.iso.org/standard/72089.html>
- International Organization for Standardization. (2021). ISO/IEC/IEEE 29119-2:2021: Software and systems engineering - Software testing - Part 2: Test processes.
<https://www.iso.org/standard/79428.html>
- International Organization for Standardization. (2021). ISO/IEC/IEEE 29119-3:2021: Software and systems engineering - Software testing - Part 3: Test documentation.
<https://www.iso.org/standard/79429.html>
- International Organization for Standardization. (2023). ISO/IEC 25010:2023: Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - Product quality model. <https://www.iso.org/standard/78176.html>
- International Organization for Standardization. (2023). ISO/IEC 42001:2023: Information technology - Artificial intelligence - Management system. <https://www.iso.org/standard/42001>
- Laravel. (2025). Laravel 12.x release notes. <https://laravel.com/docs/12.x/releases>
- Metzner, A. (2024). Systematic teaching of UML and behavioral diagrams. 2024 IEEE 36th Conference on Software Engineering Education and Training (CSEE&T).
<https://doi.org/10.1109/CSEET62301.2024.10663036>

Open Worldwide Application Security Project. (2021). OWASP Application Security Verification Standard 4.0.3. <https://owasp.org/www-project-application-security-verification-standard/>

Open Worldwide Application Security Project. (2021). OWASP Top 10:2021. <https://owasp.org/Top10/2021/>

Open Worldwide Application Security Project. (2023). OWASP API Security Top 10 - 2023. <https://owasp.org/API-Security/editions/2023/en/Ox11-t10/>

Open Worldwide Application Security Project. (2024). Web Security Testing Guide. <https://owasp.org/www-project-web-security-testing-guide/>

Oracle. (2026). MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>

PHPUnit. (2024). PHPUnit 11 documentation. <https://phpunit.de/documentation.html>

Redis. (2026). Redis documentation. <https://redis.io/docs/latest/>

Sentry. (2026). Sentry for Laravel documentation. <https://docs.sentry.io/platforms/php/guides/laravel/>

World Wide Web Consortium. (2018). Web Content Accessibility Guidelines (WCAG) 2.1. <https://www.w3.org/TR/WCAG21/>